

SMART DERMATOLOGY DIAGNOSIS USING AI – REPORT

1. PROJECT DESCRIPTION

1.1 INTRODUCTION

Skin diseases are among the most common health problems affecting individuals worldwide. Millions of people are diagnosed each year with dermatological conditions ranging from mild inflammatory disorders to life-threatening malignant lesions. Early and accurate diagnosis plays a crucial role in preventing complications and ensuring timely treatment. However, access to experienced dermatologists is often limited, particularly in rural and resource-constrained regions. This shortage leads to delayed diagnosis and improper treatment, increasing the risk of disease progression.

Recent advancements in Artificial Intelligence and Deep Learning have demonstrated significant potential in medical image analysis. Convolutional Neural Networks (CNNs) have shown remarkable capability in extracting complex hierarchical features from images and performing multi-class classification with high accuracy. In dermatology, AI-based systems can assist clinicians by providing rapid and automated analysis of clinical images.

Despite the progress in deep learning models, challenges remain in terms of robustness, generalization, and interpretability. Many existing systems rely on single-model architectures, which may suffer from overfitting and reduced stability when applied to diverse clinical datasets. Furthermore, the lack of explainability in deep learning systems creates hesitation in clinical adoption, as medical professionals require transparency in diagnostic decisions.

The project “Smart Dermatology Diagnosis Using AI” proposes an explainable ensemble deep learning framework for multi-class skin disease classification covering 19 clinically relevant categories derived from the DermNet dataset. The system integrates EfficientNet-B4 and DenseNet-121 architectures using a weighted soft-voting probability fusion strategy. Test-Time Augmentation (TTA) is incorporated to improve prediction robustness. Additionally, Grad-CAM is integrated to generate heatmap visualizations highlighting discriminative regions in the clinical image, thereby enhancing transparency and clinical trust.

The system is implemented as a web-based clinical decision support platform, enabling users to upload skin images and receive structured diagnostic reports. By combining ensemble learning, inference-time augmentation, and explainable AI, the proposed system aims to provide a reliable, transparent, and practically deployable dermatological diagnostic assistant.

1.2 EXISTING SYSTEM

In the current healthcare scenario, skin disease diagnosis primarily depends on manual examination by dermatologists. The process typically involves visual inspection, dermoscopic analysis, and sometimes biopsy procedures. Although expert diagnosis is effective, it is highly dependent on physician expertise and experience. Human diagnosis may vary between practitioners and may be influenced by fatigue, subjectivity, and limited exposure to rare cases.

With the growth of AI research, several deep learning-based skin disease classification systems have been developed. Many of these systems utilize standalone CNN architectures such as traditional CNNs, MobileNet, ResNet, or EfficientNet variants. While these models achieve moderate performance, they often face limitations. Most existing AI-based dermatology models operate as black-box systems, providing predictions without explaining the reasoning behind decisions. In medical applications, this lack of transparency reduces clinician trust and limits real-world deployment.

Furthermore, many existing systems are experimental and lack integrated web deployment, structured report generation, and database management capabilities, which are necessary for practical clinical implementation.

1.2.1 DISADVANTAGES OF EXISTING SYSTEM

- Manual diagnosis is highly dependent on dermatologist experience and may vary between practitioners.
- Limited availability of dermatologists in rural and remote areas leads to delayed diagnosis.
- Traditional diagnosis methods are time-consuming and may require additional laboratory tests.
- Single deep learning models lack robustness and may suffer from overfitting.
- Standalone architectures do not effectively handle visually similar skin disease classes.
- Most AI-based systems operate as black-box models without explainability mechanisms.

- Lack of interpretability reduces clinical trust and practical adoption.
- Many research models are not deployed as real-time web-based systems.
- Absence of structured diagnostic report generation limits documentation and follow-up.
- Poor generalization performance when tested on unseen or diverse datasets.

1.3 PROPOSED SYSTEM

The proposed system introduces an explainable ensemble deep learning framework for 19-class skin disease classification. The system integrates two heterogeneous CNN architectures, EfficientNet-B4 and DenseNet-121, to leverage complementary feature extraction capabilities.

EfficientNet-B4 employs compound scaling to balance depth, width, and resolution, enabling optimized hierarchical feature learning. DenseNet-121 uses dense connectivity to enhance gradient flow and feature reuse, improving discrimination between visually similar dermatological conditions. The outputs of both models are combined using a weighted probability fusion strategy defined as:

$$P_final = 0.65 \times P_B4 + 0.35 \times P_Dense$$

where P_B4 and P_Dense represent the Softmax probability outputs of EfficientNet-B4 and DenseNet-121 respectively.

To further improve robustness, Test-Time Augmentation (TTA) is applied during inference by generating augmented versions of input images and averaging predictions. This reduces sensitivity to spatial variations and enhances generalization. Grad-CAM is integrated to provide visual explanations by generating heatmaps that highlight image regions contributing to classification decisions. The system also generates structured PDF reports containing predicted disease class, confidence score, and heatmap visualization.

The proposed system is deployed as a web-based platform using Flask backend and SQLite database for secure data storage and retrieval.

1.3.1 ADVANTAGES OF PROPOSED SYSTEM

- Uses ensemble learning by combining EfficientNet-B4 and DenseNet-121 for improved accuracy.
- Weighted probability fusion reduces model bias and enhances prediction stability.
- Supports classification of 19 clinically relevant skin disease categories.
- Test-Time Augmentation improves robustness and generalization performance.
- Grad-CAM provides visual explanations for model predictions.
- Enhances clinical trust through explainable AI integration.
- Web-based deployment allows easy access and usability.
- Generates structured PDF diagnostic reports automatically.
- Stores patient and prediction records securely using SQLite database.
- Reduces dependency on immediate dermatologist availability.
- Improves early detection and screening capability.
- Scalable architecture suitable for future expansion and real-world deployment.

1.3.2 HARDWARE REQUIREMENTS

Processor	: Intel i5 or higher
RAM	: Minimum 8 GB (16 GB recommended for training)
Storage	: 256 GB SSD or higher
GPU	: NVIDIA GPU recommended for training
Monitor	: Standard SVGA Display
Input Devices	: Keyboard and Mouse

1.3.3 SOFTWARE REQUIREMENTS

Programming Language	: Python
Deep Learning Framework	: PyTorch
Web Framework	: Flask
Database	: SQLite
IDE	: Visual Studio Code
Training Platform	: Kaggle / Google Colab
Operating System	: Windows 10 / Linux

2. LOGICAL DEVELOPMENT

2.1 DATA FLOW DIAGRAM

Level 0

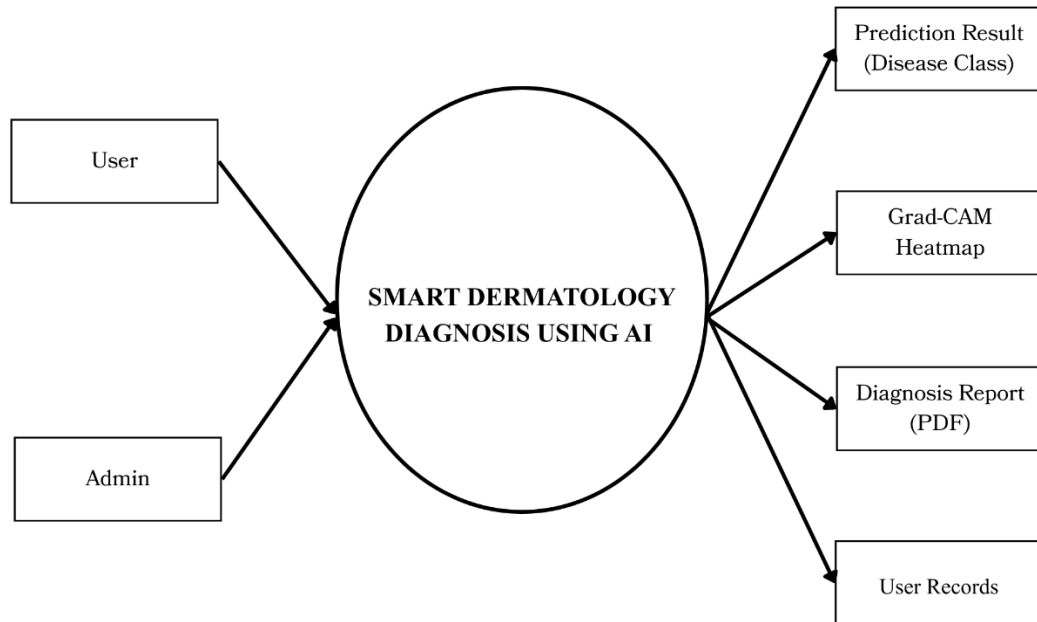


Figure 2.1 – Level 0 Data Flow Diagram of Smart Dermatology Diagnosis Using AI

Level 1

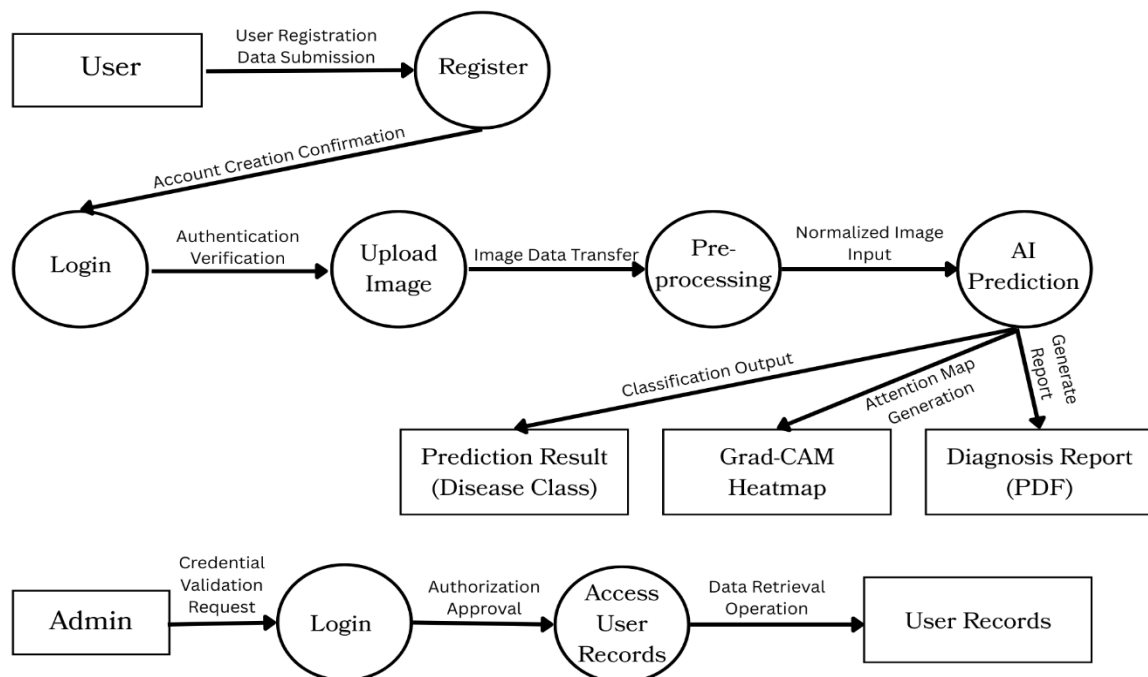


Figure 2.2 – Level 1 Data Flow Diagram of Smart Dermatology Diagnosis Using AI

2.2 ARCHITECTURAL DESIGN

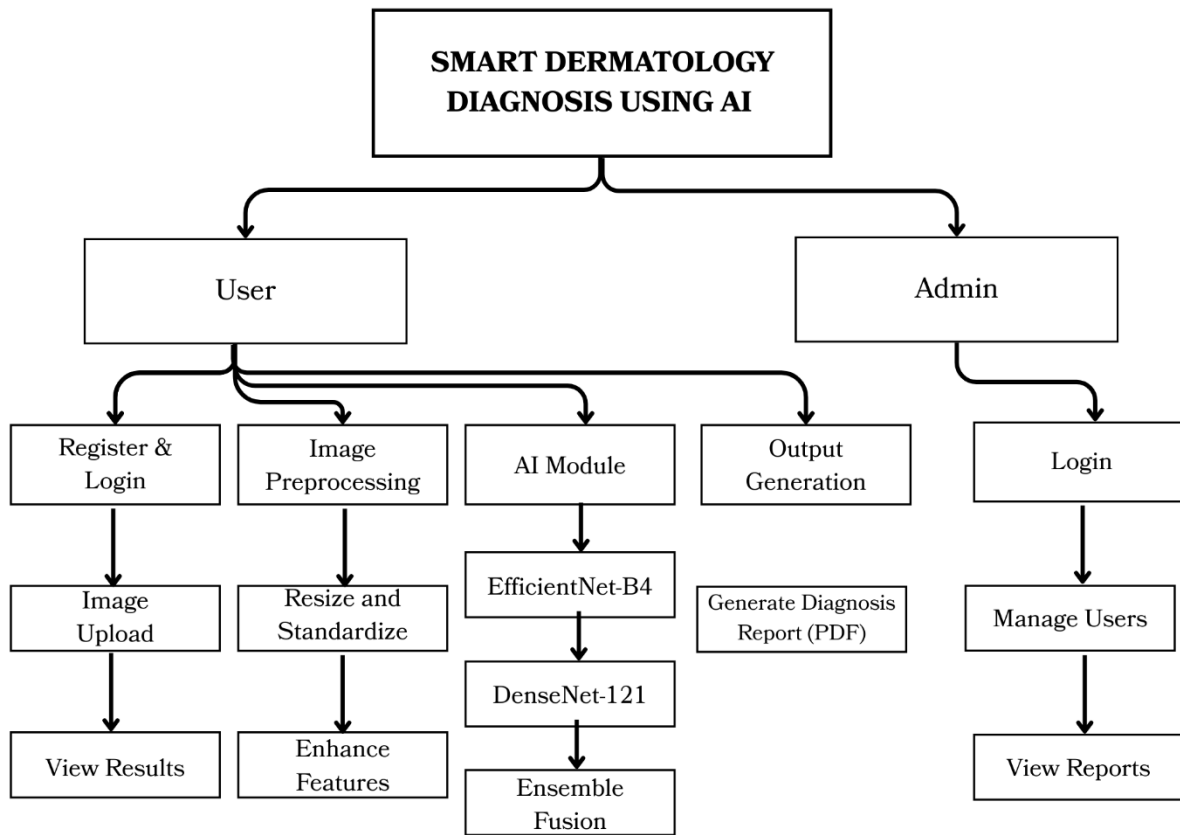


Figure 2.3 – Architectural Design of Smart Dermatology Diagnosis Using AI

3. DATABASE DESIGN & DATASET DESCRIPTION

3.1 DATA DICTIONARY

A data dictionary is a structured repository that describes the data elements used in a database system. It provides detailed information about each data field including its name, type, size, purpose, and relationship with other fields. In the Smart Dermatology Diagnosis Using AI project, the data dictionary defines the structure of tables used to store user information, login activity, and prediction results generated by the AI model. This documentation ensures consistency in database design and simplifies data management and maintenance.

S.No	Field Name	Description	Sample Data
01	id	User ID (users table primary key)	14U0001
2	username	Username for login	johndoe
3	email	Email address of the user	johndoe@gmail.com
4	password_hash	Hashed password	\$2b\$12...
5	role	Role of the user (e.g., patient, admin)	patient
6	is_verified	User email verification status	1
7	patient_name	Full name of the patient/user	John Doe
8	gender	Gender of the user	Male
9	location	General location/city of the user	New York
10	state	State of residence	NY
11	country	Country of residence	USA
12	profile_completed	Profile completion flag status	1
13	id	Location ID	100
14	user_id	Foreign key linking to user id	14U0001
15	latitude	GPS Latitude for location tracking	40.7128
16	longitude	GPS Longitude for location tracking	-74.0060
17	timestamp	Time of location update	2026-03-02 12:00:00
18	id	OTP ID	50
19	username	Username requesting the OTP	johndoe
20	otp_code	6-digit OTP code for verification	123456

21	created_at	Time the OTP was generated	2026-03-02 12:05:00
22	id	Login history record ID	200
23	username	Username that logged into the system	johndoe
24	login_time	Timestamp of the login event	2026-03-02 12:10:00
25	id	Prediction record ID	300
26	username	Username who performed the prediction	johndoe
27	disease	Predicted skin disease name	Melanoma
28	confidence	Confidence score of the prediction	98.5
29	timestamp	Time the prediction was made	2026-03-02 12:15:00
30	original_img	Filename/path of the original image	img_001.jpg
31	gradcam_img	Filename/path of the GradCAM heatmap	gradcam_001.jpg

3.2 TABLE DESIGN

In database systems, a table is a structured arrangement of data stored in rows and columns. Columns represent specific attributes of the data and are identified by unique names, while rows represent individual records. Each table may contain many rows but has a fixed set of columns. A unique identifier, often called a primary key, is used to distinguish each record within the table. The system database consists of the following primary tables:

Table: users

S.No	Field Name	Type	Constraints
1	id	INTEGER	Primary Key
2	username	TEXT	Unique
3	email	TEXT	Null
4	password_hash	TEXT	Null
5	role	TEXT	Null
6	is_verified	INTEGER	Default 1
7	patient_name	TEXT	Null
8	gender	TEXT	Null
9	location	TEXT	Null
10	state	TEXT	Null

11	country	TEXT	Null
12	profile_completed	INTEGER	Default 0

This table stores information related to registered users of the system. It typically includes details such as user ID, name, email address, password, and other profile information required for authentication and identification.

Table: login_history

S.No	Field Name	Type	Constraints
1	id	INTEGER	Primary Key
2	username	TEXT	Null
3	login_time	TEXT	Null

The login_history table records the login activity of users. It stores information such as login time, logout time, and user ID to maintain a history of system access. This helps in monitoring user activity and improving system security.

Table: predictions

S.No	Field Name	Type	Constraints
1	id	INTEGER	Primary Key
2	username	TEXT	Null
3	disease	TEXT	Null
4	confidence	REAL	Null
5	timestamp	TEXT	Null
6	original_img	TEXT	Null
7	gradcam_img	TEXT	Null

This table stores the diagnostic results generated by the AI system. It includes information such as the uploaded image, predicted disease category, prediction confidence score, and the corresponding user ID. This table helps maintain a record of past diagnostic results and allows users or administrators to review prediction history.

3.3 INTRODUCTION TO THE DATASET

In the project “Smart Dermatology Diagnosis Using AI,” a well-organized dermatological image dataset has been utilized to develop a multi-class skin disease classification system.

The dataset contains approximately 4000 clinical dermatology images, categorized into nineteen different disease classes. These images were prepared to support supervised deep learning techniques where each image is associated with a predefined disease label.

The images represent real dermatological cases captured under different clinical environments. Variations exist in skin tone, lesion structure, lighting conditions, backgrounds, and image capturing devices. Such diversity is important for ensuring that the trained model can perform effectively in real-world conditions rather than being limited to a specific image distribution. During dataset preparation, images were carefully organized, cleaned, and validated to ensure reliability and consistency for deep learning applications.

3.4 DATASET SOURCE AND ORGANIZATION

The dataset organization is inspired by publicly available dermatology image repositories that follow structures similar to the DermNet dataset format. These repositories provide categorized images representing a wide range of skin diseases. However, raw dermatological datasets often contain highly detailed subcategories that can complicate the classification process.

To simplify model training, the dataset used in this project was reorganized into a directory-based structure compatible with deep learning frameworks such as PyTorch. Each disease category is stored inside a separate folder, and the folder name acts as the class label for the images contained within it.

The dataset is further divided into training and testing sets to allow fair evaluation of the model. This separation ensures that the model is tested on images that were not used during training, thereby preventing data leakage and improving the reliability of performance evaluation. The consistent folder structure also simplifies the process of loading data and mapping class labels during model training.

3.5 TOTAL IMAGE COUNT AND CLASS DISTRIBUTION

The dataset consists of roughly 4000 dermatological images distributed across nineteen disease categories. On average, each class contains about two hundred images, although the distribution is not perfectly balanced.

Certain common skin conditions such as eczema, psoriasis, and fungal infections contain a larger number of images, while comparatively rare conditions such as urticaria, lupus-related disorders, or connective tissue diseases have fewer samples.

Class imbalance is a typical issue in medical datasets because the availability of images varies depending on disease prevalence. Such imbalance may cause deep learning models to favor classes with more samples. To address this issue, the project incorporates ensemble learning and probability fusion techniques to improve classification performance and reduce bias toward majority classes.

3.6 FINAL NINETEEN DISEASE CATEGORIES

After organizing and merging related subclasses, the dataset was finalized into nineteen dermatological categories. These categories include:

- Acne and Rosacea
- Bacterial Infection
- Benign Tumors
- Contact Dermatitis
- Eczema
- Fungal Infection
- Hair Loss
- Herpes and Sexually Transmitted Diseases
- Infestations and Bites
- Lupus and Connective Tissue Disease
- Malignant Lesions
- Melanoma and Nevi
- Nail Disease
- Pigmentation Disorders
- Psoriasis and Lichen Planus

- Systemic Disease
- Urticaria
- Vascular Disorders
- Viral Infection

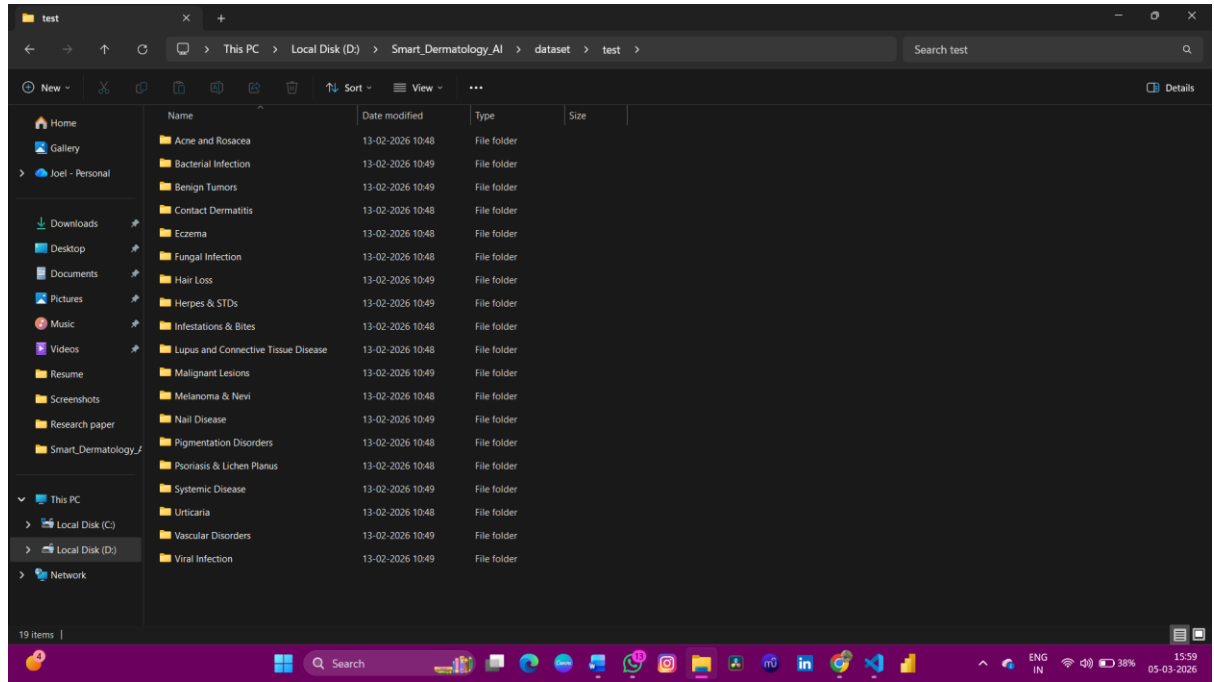


Figure 3.1 – Classes of final 19 categories

These categories were designed to represent clinically meaningful diagnostic groups rather than extremely specific pathological subclasses. This approach balances classification detail with practical diagnostic usefulness.

3.7 CLASS MERGING STRATEGY

A significant step during dataset preparation involved combining related dermatological subclasses into broader categories. Many dermatology datasets include extremely detailed subclasses, which can make model training difficult due to visual similarity between categories. For instance, several types of acne were grouped into a single category called “Acne and Rosacea.” Multiple pigmentation-related disorders were merged under “Pigmentation Disorders.” Similarly, different fungal infection variations were grouped together under “Fungal Infection.”

Conditions such as psoriasis and lichen planus were combined into one category because they often share similar visual features. Viral skin conditions were merged into a general “Viral Infection” class, and different benign growths were grouped into “Benign Tumors.”

This merging strategy helps improve class balance, reduces unnecessary label fragmentation, and minimizes confusion between visually similar subclasses. As a result, the model can learn stronger and more generalized patterns.

3.8 IMAGE CHARACTERISTICS AND VARIABILITY

The dataset images vary significantly in terms of resolution, color intensity, lighting conditions, lesion size, anatomical location, and viewing angle. Some images are captured in clinical environments using professional equipment, while others may originate from standard consumer cameras. Such variability improves the model’s ability to handle real-world dermatological images, which are rarely captured under standardized conditions. However, this diversity also introduces challenges. Differences in brightness and contrast may affect lesion visibility, while variations in skin tone may influence texture-based feature extraction. Background noise and unrelated visual elements may also affect model predictions.

3.9 DATA SPLITTING STRATEGY

To objectively evaluate the model, the dataset was divided into training and testing subsets. The training set is used for learning the model parameters, while the testing set is used to measure the model’s performance. The separation ensures that images used for testing are not included in the training data. This prevents data leakage and ensures that evaluation results accurately represent the model’s ability to generalize to unseen images. Maintaining a similar class distribution between training and testing sets further ensures reliable and fair performance measurement.

3.10 PREPROCESSING CONSIDERATIONS

Because raw images vary in size and format, preprocessing is necessary before feeding them into a deep learning model. All images are resized, which matches the input size required by EfficientNet-B4

and DenseNet-121 architectures. In addition, pixel values are normalized using ImageNet mean and standard deviation values. This normalization aligns the dataset with pretrained model parameters and improves feature extraction.

The images are then converted into tensor format, which allows efficient computation within the PyTorch deep learning framework. These preprocessing steps ensure consistent input representation and stable model training.

3.11 DATASET LIMITATIONS IN DERMATOLOGICAL IMAGE CLASSIFICATION

Dermatological image classification presents several challenges. Many skin conditions share similar visual features such as redness, scaling, inflammation, or pigmentation changes, which leads to high similarity between different disease classes. At the same time, the same disease can appear differently depending on factors such as severity, body location, and patient skin tone. This creates significant variation within the same class. Another major challenge is the limited availability of labeled medical images, especially for rare diseases.

Privacy concerns and ethical restrictions often limit access to large medical datasets. These factors increase the complexity of building accurate multi-class classification models. To address these difficulties, the project uses an ensemble learning approach, combining multiple deep learning models to capture complementary feature patterns.

3.12 SUITABILITY OF THE DATASET FOR DEEP LEARNING

Despite the inherent challenges, the dataset used in this project is well suited for deep learning-based skin disease classification. The dataset contains approximately 4000 images across nineteen disease categories, which provides sufficient information for learning meaningful features. The class merging strategy improves label stability and reduces excessive fragmentation, while the diversity of images enhances the model's ability to generalize to real-world data. The organized folder structure also supports automated supervised learning workflows. Overall, careful dataset preparation, restructuring, and validation play a crucial role in improving the robustness, reliability, and clinical relevance of the AI-based dermatology diagnosis system.

4. PROGRAM DESIGN

4.1 OVERALL SYSTEM WORKFLOW

The Smart Dermatology Diagnosis Using AI system is architected as a modular, scalable, and intelligent web-based clinical decision support platform designed to assist in automated multi-class skin disease classification. The system is structured to bridge the gap between conventional dermatological assessment and modern artificial intelligence-based diagnostic automation. It integrates secure authentication, image acquisition, preprocessing, ensemble deep learning inference, explainable AI visualization, structured report generation, and database-driven record management into a unified and logically sequenced workflow.

The workflow of the system is sequential yet modular in nature. Each module performs a well-defined responsibility, ensuring separation of concerns, ease of maintenance, and scalability for future enhancements. The operational pipeline begins with user authentication, where credentials are verified against stored database records. Upon successful authentication, the user gains access to the system's diagnostic interface and uploads a dermatological clinical image. The uploaded image undergoes validation to ensure format compatibility and secure storage within the server environment.

After validation and storage, the image is forwarded to the preprocessing pipeline, where resizing, normalization, and tensor transformation are performed to ensure compatibility with the deep learning models. The preprocessed image is then passed into the deep learning inference engine, which executes parallel evaluation using EfficientNet-B4 and DenseNet-121 architectures. Both networks independently compute probability distributions across the nineteen predefined skin disease categories.

The probability outputs are then combined using weighted soft voting in the ensemble fusion module. A final predicted disease class and associated confidence score are computed from the fused probability vector. To enhance transparency, the Grad-CAM module generates a heatmap highlighting discriminative regions that influenced the classification decision. A structured PDF diagnostic report is then dynamically generated and archived in the database. The system ensures that prediction records are linked to user identifiers, allowing both users and administrators to access stored reports securely. This comprehensive workflow ensures robustness, reproducibility, scalability, and maintainability of the overall system.

4.2 USER AUTHENTICATION AND ACCESS CONTROL MODULE

This module plays a critical role in maintaining system security and protecting sensitive medical data. Since the system handles dermatological images and prediction reports, controlled access is essential to prevent unauthorized usage and data breaches.

During the registration phase, the system collects user information such as username, email address, password, and role specification. Input validation mechanisms ensure that the email follows a valid format, the password satisfies minimum strength requirements, and the username does not already exist in the database. To prevent security vulnerabilities, passwords are securely hashed before being stored. This ensures that even in the event of database exposure, plaintext credentials are not compromised.

During login, the system retrieves stored user records and performs password hash comparison to validate credentials. Upon successful authentication, a secure session token is generated and maintained throughout the user's interaction with the system. If authentication fails, access is denied, and an error message is displayed. The system includes validation checks to minimize repeated unauthorized attempts and improve security reliability.

Role-based access control is implemented to differentiate between regular users and administrators. Users are permitted to upload images, view predictions, and download reports, whereas administrators are granted extended privileges such as viewing all prediction records, monitoring system usage, and managing user accounts. This hierarchical access structure enhances system governance and data security.

4.3 IMAGE ACQUISITION AND VALIDATION MODULE

The Image Acquisition and Validation Module acts as the primary interface between the user and the artificial intelligence system. It ensures that the images provided for diagnosis are properly formatted and securely managed. When a user uploads a dermatological image, the system verifies that the file format is supported. Acceptable formats include standard image types such as JPG, JPEG, and PNG. This validation prevents execution of unsupported or potentially harmful files. The system also ensures that corrupted or incomplete files are not processed.

Once validated, the image is assigned a unique image identifier to maintain traceability. The file is stored in a secured server directory, and its storage path is recorded in the database along with metadata such as upload timestamp and associated user identifier. This structured storage mechanism ensures organized record management and facilitates future retrieval for review or administrative purposes.

4.4 IMAGE PREPROCESSING PIPELINE

Preprocessing is a crucial step in preparing raw clinical images for deep learning inference. Since convolutional neural networks require standardized input dimensions and normalized pixel distributions, the preprocessing pipeline ensures that all images meet model input requirements. The first step in preprocessing involves resizing the image to 224×224 pixels to match the expected input resolution of both EfficientNet-B4 and DenseNet-121. This resizing operation preserves essential spatial characteristics while ensuring uniform dimensionality.

Following resizing, pixel intensity normalization is performed using predefined mean and standard deviation values consistent with ImageNet pretrained weights. Normalization scales pixel values into a standardized range, which improves numerical stability during inference and reduces sensitivity to lighting variations.

The image is then converted into a three-channel tensor representation compatible with PyTorch's computational framework. During inference, images may be grouped into batches to optimize memory allocation and computational throughput. This structured preprocessing pipeline enhances model stability, reduces noise influence, and ensures consistent performance across varied image conditions.

4.5 DEEP LEARNING INFERENCE ENGINE

The Deep Learning Inference Engine constitutes the core analytical component of the system. It integrates two independently fine-tuned convolutional neural networks to perform robust multi-class classification across nineteen dermatological categories.

4.5.1 EfficientNet-B4 Architecture

EfficientNet-B4 is a scaled convolutional neural network, which abacus-scales convolutional network depth, width, and input resolution in a proportionate manner. The balanced scaling method

guarantees maximum usage of the parameters and is also computationally efficient. In contrast to the ad hoc way network depth or width are added in traditional architectures, EfficientNet uses a systematic scaling methodology, which scales the representational power of their architectures without any embarrassment in the form of unnecessary parameter inflation.

The architecture is made of Mobile Inverted Bottleneck Convolution blocks that have depthwise separable convolutions and squeeze-and-excitation mechanisms. These elements complement discrimination of features and decrease redundancy in computation. The compound scaling strategy allows EfficientNet-B4 to learn subtle dermatological textures like lesion texture, color changes, structural abnormality, and boundary misshapeness. Its balanced architecture minimizes overfitting and offers efficient computations besides offering good classification.

4.5.2 DenseNet-121 Architecture

DenseNet-121 has dense connectivity in which each layer receives all the inputs of the previous layers. Such design helps in finding a better gradient flow and reuse of features throughout the network. DenseNet is capable of improving the information flow by combining feature maps of previous layers, thus addressing the vanishing gradient issue.

It is an architectural design made up of several blocks of dense blocks that are isolated with transition layers. Multi-scale features are incorporated in each dense block, and the network is able to pick up subtleties of dermatological differences between various levels of abstraction. The reuse of features minimizes redundancy and helps in making the parameters more effective. Within the scope of dermatological classification, DenseNet-121 serves to supplement EfficientNet-B4 with other perspectives of features and thus increases diversity and strength in the ensemble.

4.6 ENSEMBLE FUSION STRATEGY

The ensemble fusion approach is to merge the results of EfficientNet-B4 and DenseNet-121 to enhance the consistency of the classifications. Individual model predictions are prone to bias or variance based on the conditions of training and the properties of the data. To circumvent these limitations, ensemble learning combines a number of model viewpoints.

The two networks generate Softmax probability vectors during inference. Weighted soft voting is then done to compute the final probability vector in which a greater weight is weighted out to the model that has illustrated an excellent validation score. Moreover, confidence-sensitive weighting can be used based on the assessment of maximum scores of probabilities. The last predicted class is the one with maximum of the fused probability distribution. This approach would minimise the risk of misclassification, increase robustness, and extend generalization to a variety of both dermatological diseases.

4.7 TEST-TIME AUGMENTATION STRATEGY

The Test-Time Augmentation enhances the stability of inference by producing slight differences of the input image during prediction. Horizontal flipping, rotation slightly, and brightness augmentations are done. All the augmented versions are taken through the ensemble model and the probability outputs are averaged to obtain the final prediction.

The method ensures that the effect of spatial changes is less sensitive and that prediction stability is improved without having to re-train the model. Test-Time Augmentation enhances resilience and augments classification reliability in practice.

4.8 GRAD-CAM EXPLAINABILITY SYSTEM

Grad-CAM is more interpretable as it produces gradient-based localization maps, which allow viewing the discriminative zones of the image. The final convolutional layer is calculated to compute gradients of the predicted class with respect to feature maps of that layer. These gradients are added to obtain importance weights that are aggregated with feature maps to obtain a localization heatmap.

The heatmap is overlaid on the initial image to graphically show the influential regions. Such visualization will allow clinicians to check the logic of prediction, determine the areas of suspicious lesions, and develop confidence in the use of AI in diagnosis.

4.9 DIAGNOSTIC REPORT GENERATION

Structured PDF reports with patient information, image reference measures, predicted disease type, confidence value, Grad-CAM and prediction time are dynamically generated by the system. The process of report generation is automated and structured in a manner that can be read by the clinicians. Created reports are stored in the database and can be downloaded to facilitate records and medical visitation.

4.10 DATABASE DESIGN AND DATA FLOW

The system makes use of SQLite as a relational database management system which is lightweight, file based and lightweight in the storage of data in a structured and reliable manner. SQLite was chosen because of its simplicity, a low configuration level, and an easy integration with a Flask based web application. The database will store and process all vital information about the system, such as user credentials, uploaded image metadata, prediction outputs and created diagnostic reports. The database version has a distinct table structure of users, images, predictions and report table. Every table has got primary keys to distinctively identify records. Relational integrity in tables is determined by foreign key constraints.

As an example, every image record has a single user identification by reference to a user that is represented by a user identifiable (user_id) and each prediction record has a single image identifiable (image_id). Likewise, every report generated is linked to a prediction record such that the diagnostic results are traced. This type of design is a relational design that maintains consistency of data and eliminates orphans. It also permits effective search of past diagnostic history, so the user can gain access to historical records and administrator to keep track of the system usage. Indexing which is structured would enhance performance of queries and promote scalability of data management as the records to be stored escalate.

4.11 FLASK BACKEND INTEGRATION

Flask framework is an intermediate layer between the frontend user interface and deep learning inference engine and the database system. Flask will process HTTP requests and send the actions of users to the necessary back-end functions as well as communicate between application elements.

With an image, a user sends it, the request is converted into the Flask that validates the input, saves the file safely, initiates preprocessing, and runs the AI engine inference processing and retrieves the output of the predictions made. After that it stores findings in the database and converts the answer to user interface. Flask also handles the session management as well as role-based authentication which means that specific functions are accessible only to authorized users.

Flask allows the easy addition of artificial intelligence bits into a web-based environment by using its modular routing structure feature. It serves as the coordination layer that facilitates the flow of data between user interaction, machine learning models and long-term storage.

4.12 SYSTEM PERFORMANCE CONSIDERATIONS

The system is designed to operate efficiently on CPU-based hardware environments, ensuring accessibility without requiring high-end computational infrastructure. Memory efficiency is maintained through batched image processing and optimized tensor handling during inference. Lazy model loading techniques prevent repeated initialization of deep learning models, thereby reducing computational overhead.

The modular architecture allows seamless migration to GPU-based environments for accelerated inference if required. Additionally, the system can be extended to cloud-based deployment platforms for improved scalability and remote accessibility. Efficient resource management and streamlined processing pipelines minimize latency, enabling near real-time diagnostic responses. This performance optimization ensures that the system remains responsive and practical for clinical or academic use.

4.13 ERROR HANDLING AND VALIDATION

To ensure system reliability and operational stability, robust error handling and validation mechanisms are implemented throughout the application. During authentication, invalid login attempts are detected and handled securely to prevent unauthorized access. Input validation mechanisms verify file formats and prevent unsupported or corrupted image uploads.

The system also includes checks for missing image inputs, invalid file paths, and model loading failures. Database transactions are protected using validation controls to prevent incomplete or

inconsistent data entries. In the event of an unexpected runtime error, appropriate exception handling ensures that the application does not crash abruptly and instead provides meaningful feedback to the user. These validation and fault-tolerance mechanisms enhance system robustness, reduce downtime, and maintain data integrity, ensuring smooth and reliable operation of the intelligent diagnostic platform.

5. TESTING

In the Smart Dermatology Diagnostics Using AI system, testing plays a very important role in the process of its development. It guarantees that the system will be accurate, reliable and efficient with their operating conditions. The major aim of testing is checking and removing errors, checking functionality correctness, and ensuring that the system is fulfilling the given requirements.

The proposed system includes several functionalities such as the user authentication, image pre-processing, inferencing with EfficientNet-B4 and DenseNet-121 as well as the ensemble fusion, Grad-Cam visualization, as well as production of PDF reports. Considering the fact that the system includes not only software modules but also machine learning models, extensive testing was critical to allow the system to operate in a seamless manner and to be able to diagnostically perform.

Testing was conducted in various levels such as the unit testing, integration testing, system testing and validation testing. The modules were tested separately and later together to test the overall system performance. Accuracy, precision, recall and the F1-score were model evaluation metrics that were employed to evaluate classification effectiveness on 19 categories of skin diseases. Through the testing procedure, it was established that the system has enabled it to make uniform predictions, generate elucidable heatmaps, save information in the correct manner and present well-organized diagnostic reports without experiencing any failure during the running of the program.

5.1 UNIT TESTING

During the development of the Smart Dermatology Diagnosis Using AI system, each individual module was tested separately to ensure that it functions correctly without affecting other components. The application was executed multiple times to verify data flow, validation mechanisms, and compatibility across different execution scenarios. Errors identified during testing were corrected immediately to maintain system stability and reliability.

5.1.1 Test Case 1

Module Name: Patient Details

The Patient Details module is used to collect and store essential patient information before accessing the AI diagnostic system. In this module, the user is required to enter Patient Name, Gender,

City/Area, Country, and State. These details are stored in the user database and are later used for maintaining prediction records and report generation.

The image displays two screenshots of a web application titled "Smart Dermatology Diagnosis using AI".

The top screenshot shows the "Complete Your Profile" page. It prompts the user to "Please provide your details to continue using Smart Dermatology Diagnosis". The form includes the following fields:

- Patient Name ***: Text input with "Smith" entered.
- Gender ***: Dropdown menu with "Male" selected.
- City/Area ***: Text input with "Nagercoil" entered.
- Country ***: Dropdown menu with "India" selected.
- State ***: Dropdown menu with "Tamil Nadu" selected.

A "Complete Profile" button is located at the bottom of the form.

The bottom screenshot shows the "AI Skin Diagnosis" page. At the top, a message states "Profile completed successfully!". Below this, the page features a large dashed box for image upload. Inside the box, there is a cloud icon with an upward arrow and the text "Click to Upload Image". Below the text, it specifies "Supported formats: JPG, PNG, JPEG".

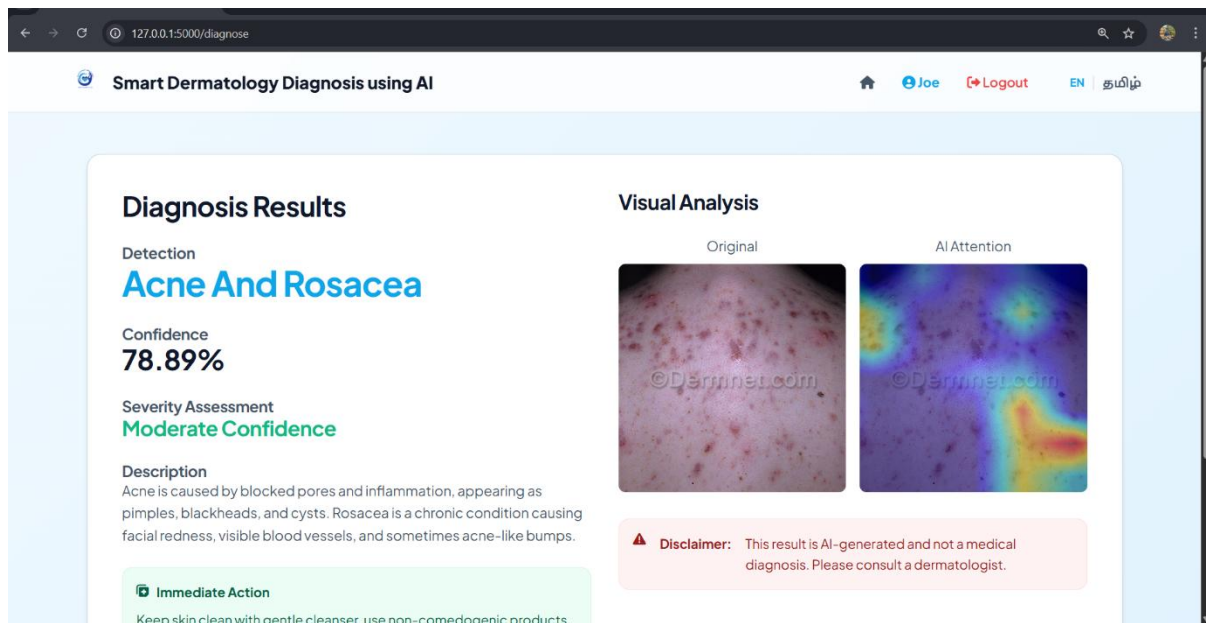
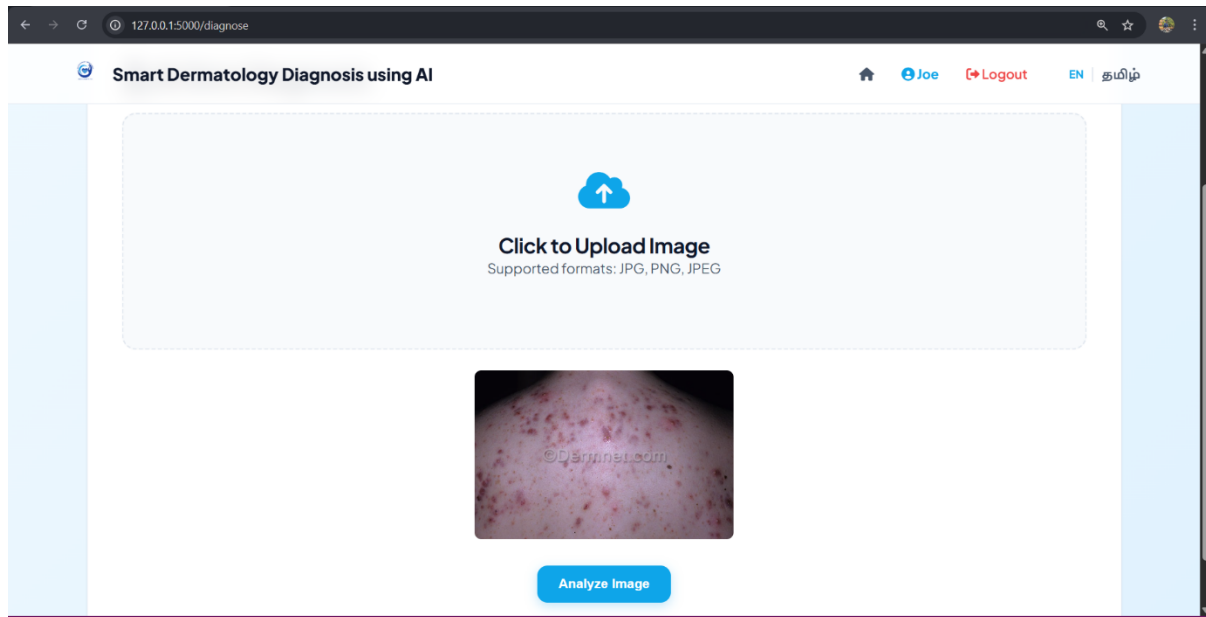
Testing Done

All required fields were filled with valid data and submitted successfully. The data was stored correctly in the database, and the confirmation message "Profile completed successfully" was displayed. The form was also tested with missing mandatory fields to ensure validation worked properly. The system prevented submission until all required fields were completed.

5.1.2 Test Case 2

Module Name: Disease Skin Image Upload

The Disease Skin Image Upload module is used to upload clinical skin images for AI-based disease classification. In this module, the user selects an image file in supported formats such as JPG, PNG, or JPEG. After uploading, the image is forwarded to the preprocessing module for further analysis.



Testing Done

Valid image files were uploaded successfully and displayed correctly on the interface. The “Analyze Image” button initiated the AI prediction process without errors. Unsupported file formats were tested to ensure they were rejected. The uploaded image was successfully passed to the preprocessing and deep learning inference module without data loss.

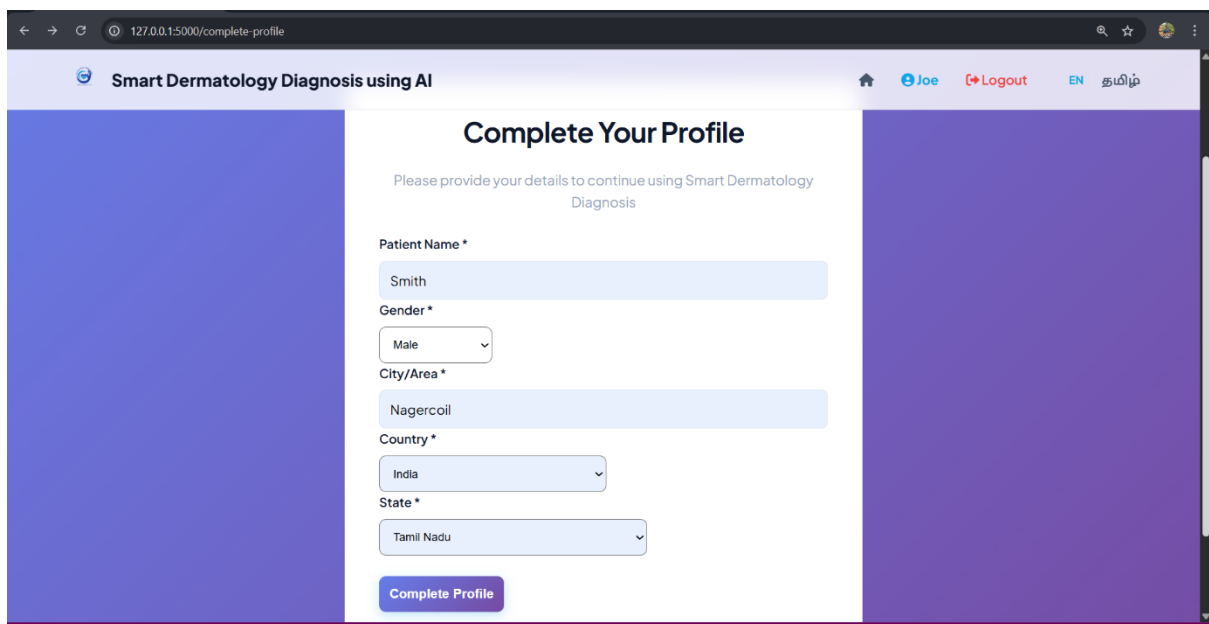
5.2 INTEGRATION TESTING

Integration testing was performed to verify the interaction between multiple modules of the Smart Dermatology Diagnosis Using AI system. The objective of this testing phase is to ensure that data flows correctly from one module to another without loss, mismatch, or processing errors. The system was executed multiple times to validate that patient details, image upload, AI inference, heatmap generation, and PDF report creation operate together as an integrated workflow.

5.2.1 Test Case 1

Module Name: Patient Details + Report with Patient Details

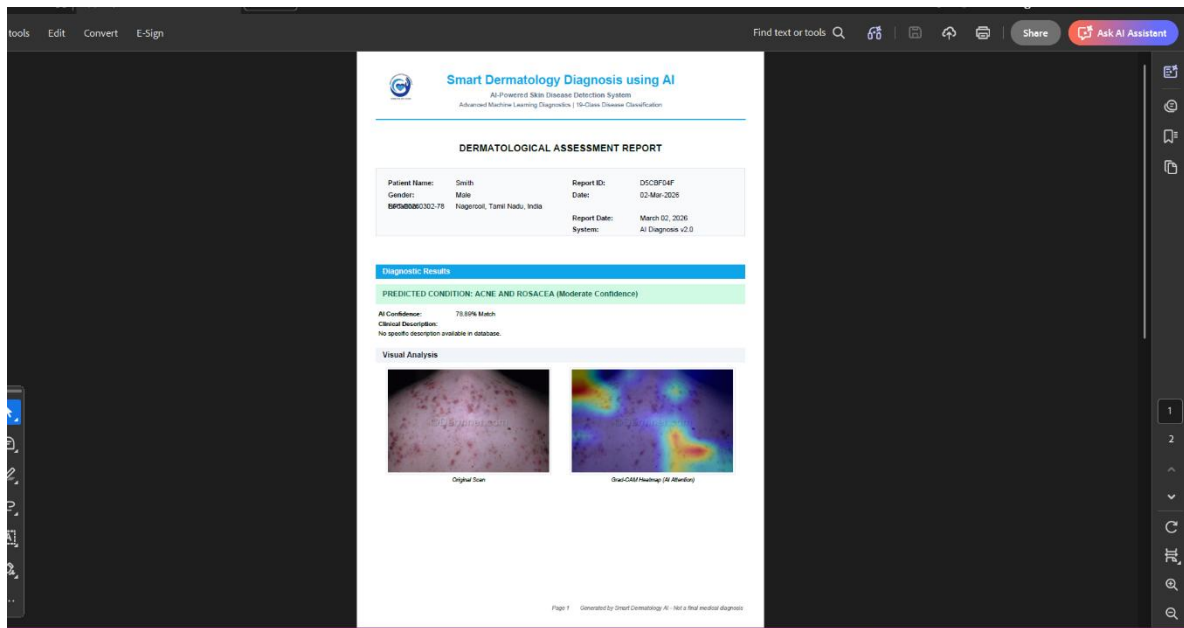
This integration verifies whether the patient profile information is properly linked with the AI diagnosis module. After completing the patient profile, the system should redirect the user to the image upload interface and associate the uploaded image with the corresponding patient record.



The screenshot shows a web browser window with the URL `127.0.0.1:5000/complete-profile`. The page title is "Smart Dermatology Diagnosis using AI". The user is logged in as "Joe" and can click "Logout" or switch to "தமிழ்" (Tamil). The main heading is "Complete Your Profile". Below it, a message says "Please provide your details to continue using Smart Dermatology Diagnosis". The form contains the following fields:

- Patient Name ***: Text input with "Smith" entered.
- Gender ***: Dropdown menu with "Male" selected.
- City/Area ***: Text input with "Nagercoil" entered.
- Country ***: Dropdown menu with "India" selected.
- State ***: Dropdown menu with "Tamil Nadu" selected.

A "Complete Profile" button is located at the bottom of the form.



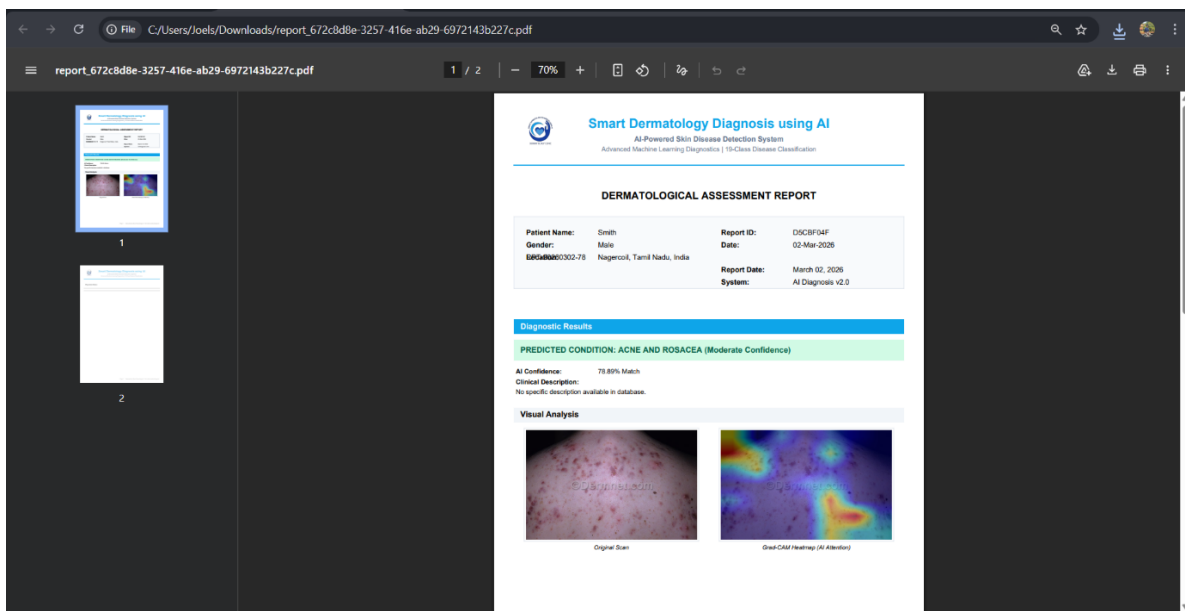
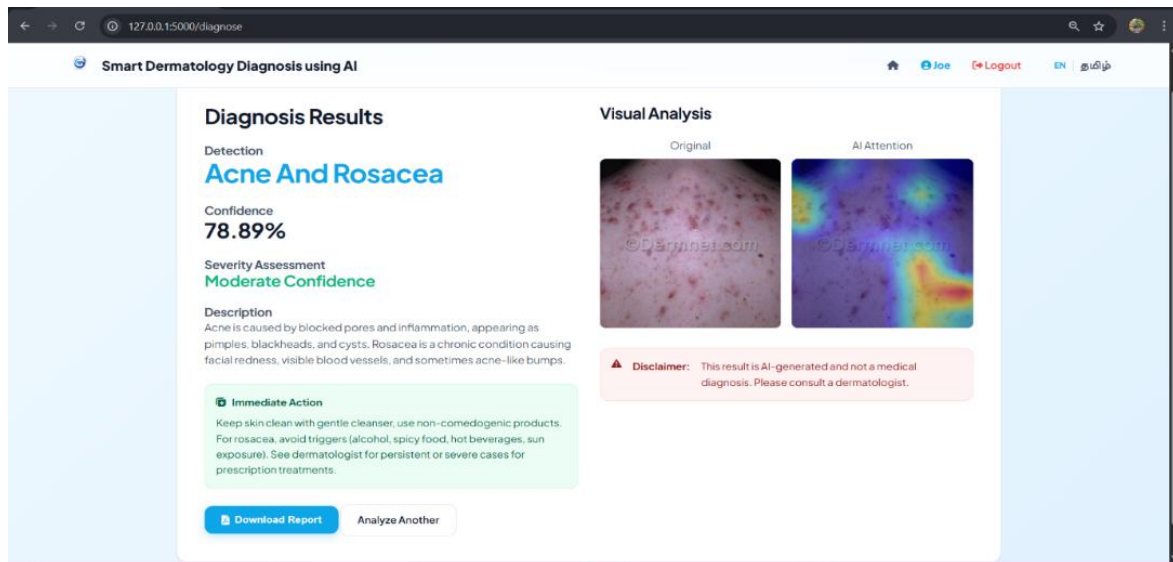
Testing Done

The patient profile was completed successfully, and the system displayed the confirmation message. After submission, the user was redirected to the AI Skin Diagnosis page. A valid skin image was uploaded and analyzed. The system correctly linked the prediction result with the stored patient details without any data mismatch in the pdf. The workflow from profile completion to diagnosis result generated as pdf was executed smoothly.

5.2.2 Test Case 2

Module Name: AI Prediction + Grad-CAM + PDF Report Generation Integration

This integration verifies whether the AI prediction module properly interacts with the Grad-CAM visualization module and the PDF report generation module. Once the model generates the predicted class, the heatmap and structured report should be created automatically.



Testing Done

A clinical image was uploaded and processed through EfficientNet-B4 and DenseNet-121 ensemble inference. The predicted disease class and confidence score were displayed successfully. The Grad-CAM heatmap was generated and aligned with the original image. Upon clicking the download option, the system generated a structured dermatological assessment report in PDF format containing patient details, prediction result, confidence score, and visual analysis. No integration errors were observed.

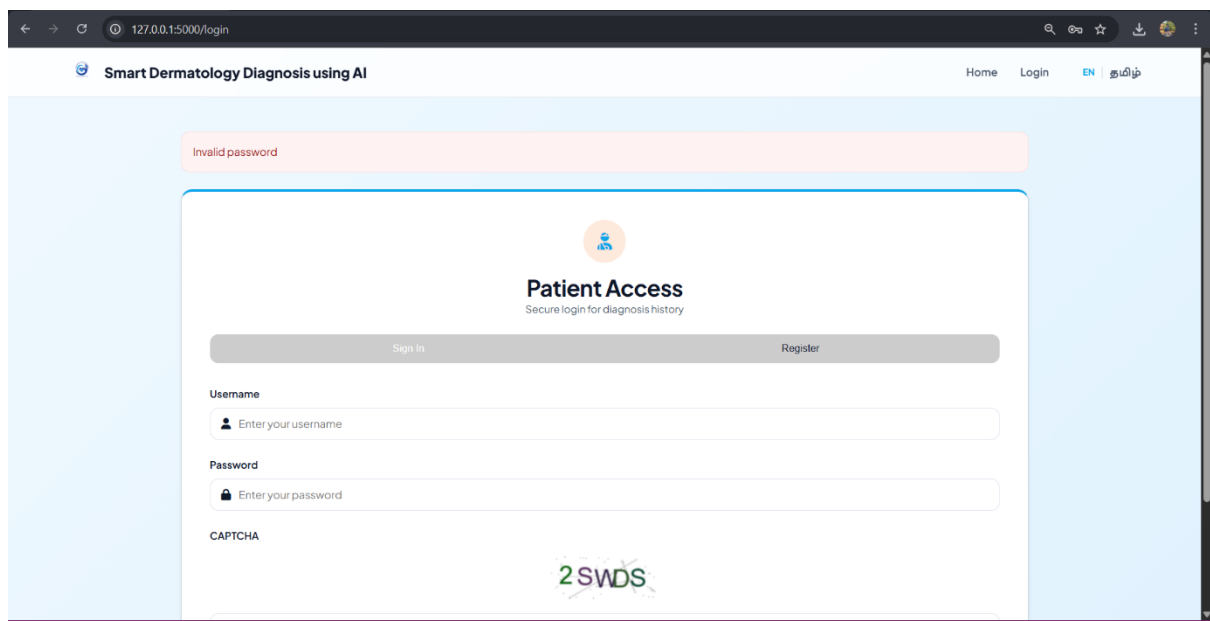
5.3 VALIDATION TESTING

Validation testing was carried out to ensure that the Smart Dermatology Diagnosis Using AI system correctly validates user inputs and prevents invalid or unauthorized data from being processed. The purpose of this testing phase is to verify that appropriate error messages are displayed and that the system enforces input constraints before allowing further operations. This testing ensures data integrity, security, and reliability of the application.

5.3.1 Test Case 1

Module Name: Login Authentication Validation

The Login module is responsible for authenticating registered users before granting access to the AI diagnostic system. It verifies the correctness of username and password credentials entered by the user.



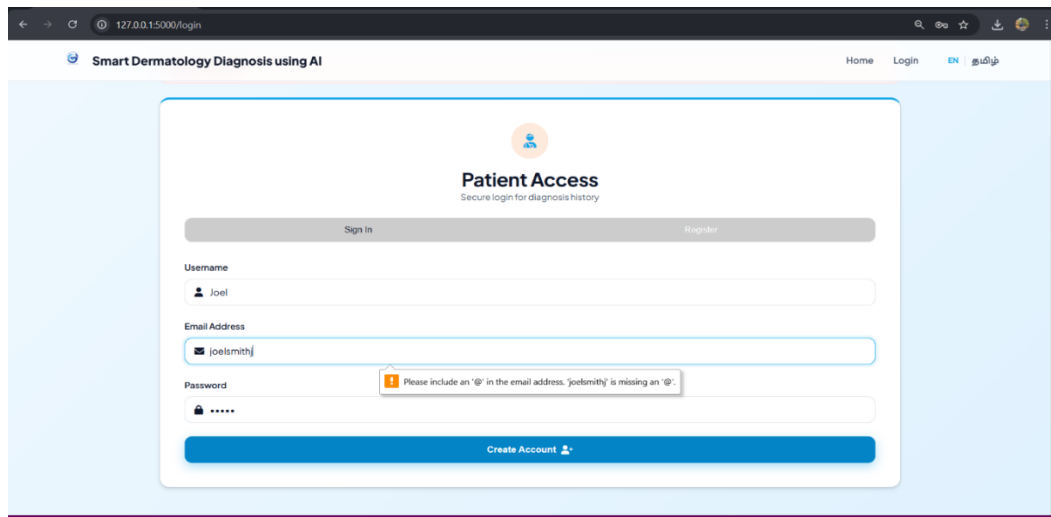
Testing Done

An incorrect password was entered while attempting to log in with a valid username. The system validated the credentials and displayed the error message “Invalid password.” The login attempt was rejected, and access to the system was denied. This confirms that unauthorized users cannot access the system without valid credentials.

5.3.2 Test Case 2

Module Name: Email Format Validation During Registration

The Registration module validates the format of the email address entered by the user while creating a new account. The system ensures that the email contains the required “@” symbol and proper domain format.



The screenshot shows a web browser window with the URL 127.0.0.1:5000/login. The page title is "Smart Dermatology Diagnosis using AI". The main heading is "Patient Access" with the subtitle "Secure login for diagnosis history". There are two tabs: "Sign In" and "Register". The "Register" tab is active. The form fields are: Username (Joel), Email Address (joelsmith), and Password (*****). A validation message is displayed: "Please include an '@' in the email address. 'joelsmith' is missing an '@'". A "Create Account" button is at the bottom.

Testing Done

An invalid email address without the “@” symbol was entered during registration. The system displayed the validation message indicating that the email format is incorrect. The account creation process was blocked until a valid email address was provided. This confirms that the system prevents invalid data from being stored in the database.

5.4 SYSTEM TESTING

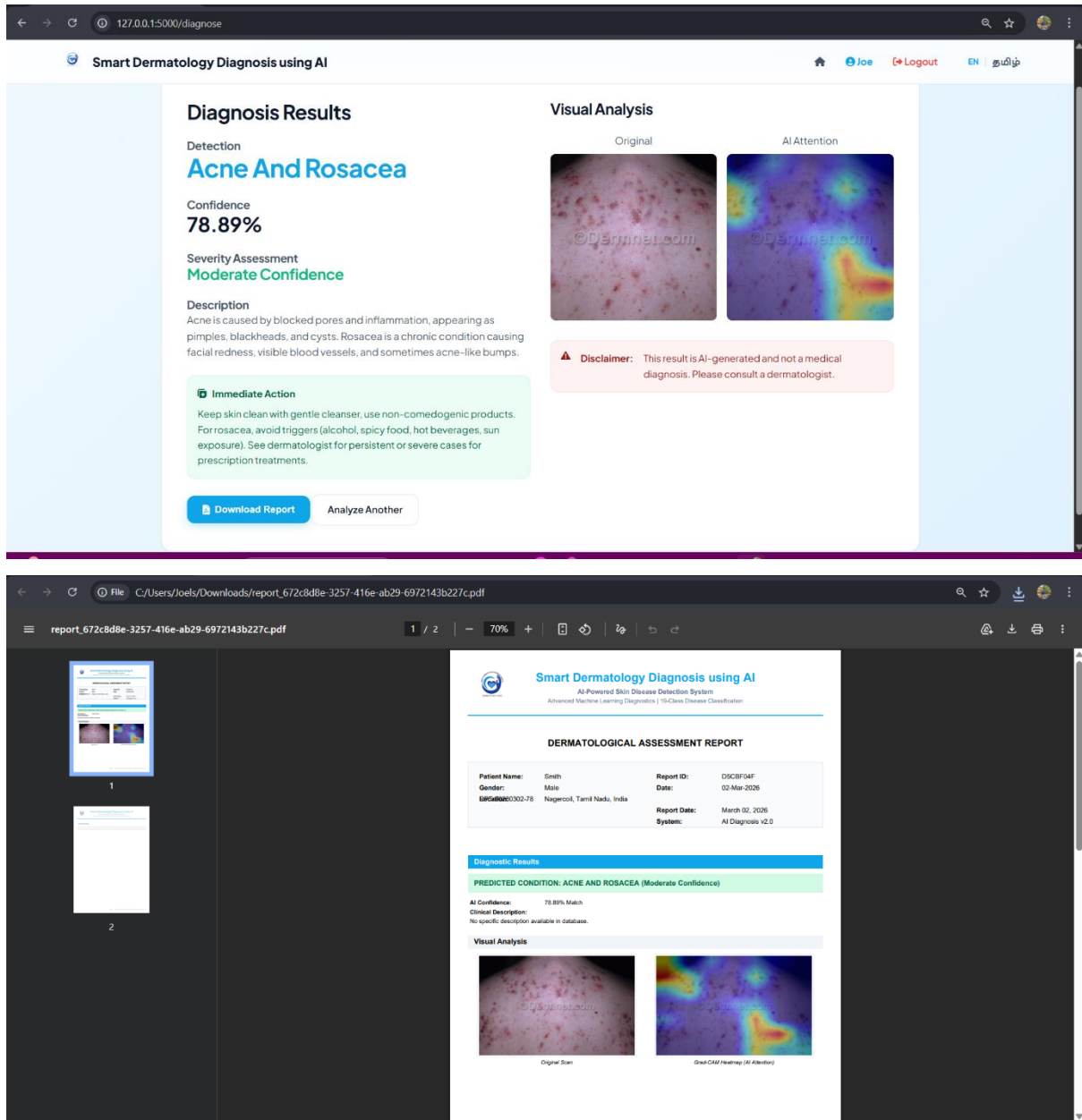
System testing was conducted to evaluate the complete Smart Dermatology Diagnosis Using AI system as a fully integrated unit. The objective of this testing phase is to verify whether all modules operate together correctly in a real-time execution environment. Unlike unit and integration testing, system testing validates the entire workflow from user login to final report generation.

The system was executed multiple times to ensure stability, accuracy, and seamless interaction between authentication, profile management, image upload, preprocessing, deep learning inference, Grad-CAM visualization, PDF report generation, and database storage modules.

5.4.1 Test Case 1

Module Name: Complete AI Diagnosis Workflow

This test case verifies the full end-to-end functionality of the application, starting from user access to final disease prediction and report generation.



Testing Done

A registered user logged into the system successfully and accessed the AI diagnosis module. The patient profile was completed, and a valid clinical skin image was uploaded. The image was processed

through the preprocessing module and forwarded to the deep learning inference module, where EfficientNet-B4 and DenseNet-121 ensemble models generated the predicted disease class and confidence score. The Grad-CAM heatmap was generated and displayed alongside the original image. Upon clicking the download option, the system generated a structured dermatological assessment report in PDF format containing patient details, predicted condition, confidence score, and visual analysis. The entire workflow executed without runtime errors or data loss.

5.5 PERFORMANCE EVALUATION

Performance evaluation was conducted to analyze and compare the effectiveness of different deep learning models used for skin disease classification. The evaluation metrics considered include Accuracy and F1-Score, which are critical indicators for medical image classification tasks.

Accuracy measures the overall correctness of predictions, whereas F1-Score provides a balanced measure of Precision and Recall, especially important for handling class imbalance in multi-class dermatological datasets.

5.5.2 F1-Score Analysis (Class-wise Performance)

Module Name: Multi-Class F1 Score Evaluation

This test case evaluates the class-wise performance of the proposed model using the F1-score metric across all 19 skin disease categories.

Testing Done

The F1-score was computed using:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The results indicate strong performance across major disease categories such as:

- Acne – 0.89
- Melanoma – 0.78
- Urticaria – 0.80

- Benign – 0.76

The proposed system maintains consistent F1-scores across most categories, indicating balanced classification performance without severe bias toward any specific class.

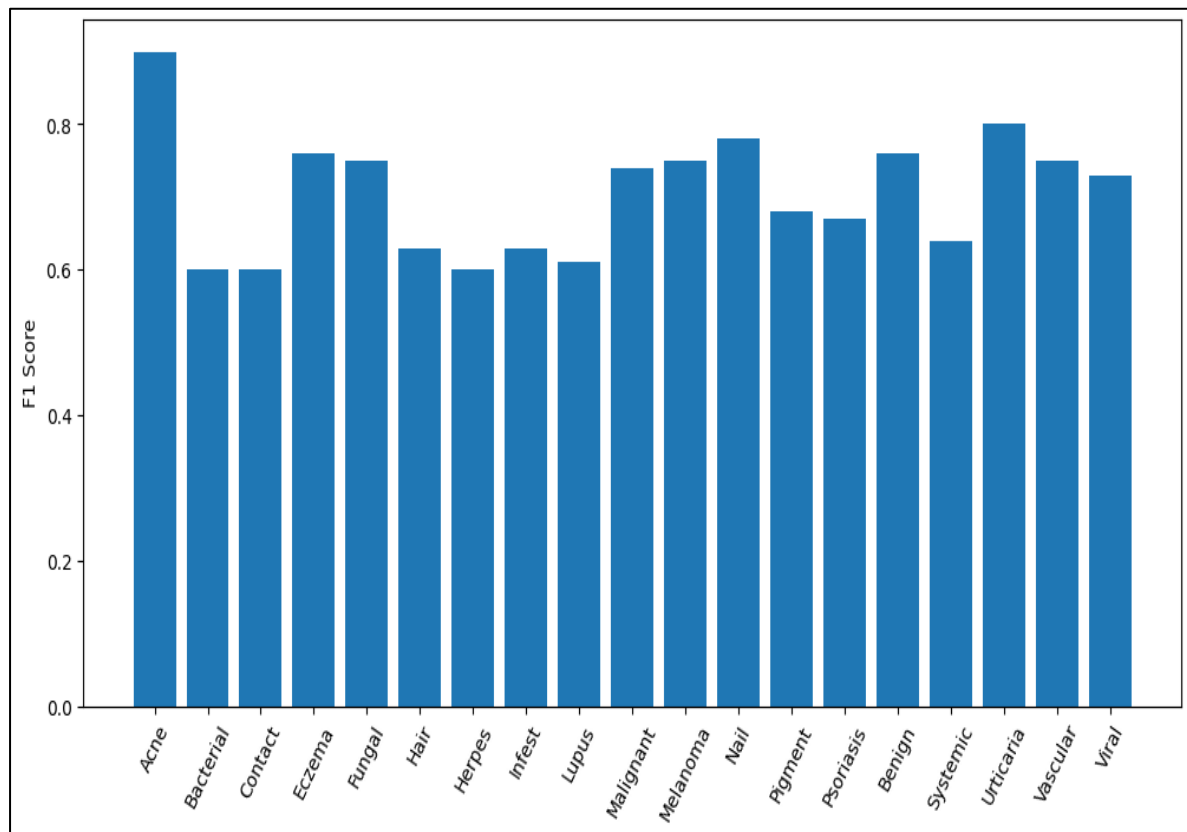


Figure 5.2: Class-wise F1 score performance

6. CONCLUSION

6.1 SUMMARY

The project titled “Smart Dermatology Diagnosis Using AI” successfully demonstrates the practical implementation of deep learning techniques for automated multi-class skin disease classification within a structured web-based clinical decision support system. The system is designed to address the challenges associated with traditional dermatological examination, particularly in scenarios where specialist availability is limited. By integrating artificial intelligence into a user-friendly digital platform, the project bridges the gap between medical expertise and technological advancement.

The system incorporates multiple interconnected modules including secure user authentication, structured image acquisition, standardized preprocessing, ensemble deep learning inference, explainable AI visualization through Grad-CAM, automated PDF report generation, and relational database management. Each module is designed with modular separation and logical independence, ensuring maintainability, scalability, and ease of future enhancement. The layered architecture allows seamless interaction between the frontend interface, backend processing engine, and AI inference components.

A major strength of the system lies in the implementation of an ensemble deep learning model combining EfficientNet-B4 and DenseNet-121 architectures. EfficientNet-B4 contributes compound-scaled feature extraction capable of capturing fine-grained dermatological patterns, while DenseNet-121 enhances feature reuse and gradient propagation through dense connectivity. The ensemble weighted fusion mechanism integrates probability outputs from both models, improving classification robustness and reducing model-specific bias. This dual-model approach enhances generalization performance across diverse skin disease categories.

The integration of Grad-CAM visualization significantly enhances model interpretability. By generating heatmaps that highlight discriminative regions influencing the prediction, the system transforms a traditionally black-box deep learning model into a more transparent and clinically interpretable decision support tool. This feature strengthens user confidence and supports potential real-world clinical adoption.

The system supports classification across nineteen dermatological conditions, demonstrating its capability to handle complex multi-class medical image analysis tasks. The structured database design

ensures secure storage of user information, image metadata, prediction results, and generated reports. Role-based access control further enhances system security by separating user and administrative privileges.

From a software engineering perspective, the project emphasizes modular design, scalability, reliability, and performance optimization. The backend integration using Flask ensures smooth communication between AI components and web services. Error handling mechanisms and validation layers improve system robustness and operational stability.

Overall, the project establishes a comprehensive foundation for AI-assisted dermatological diagnosis systems. It validates the feasibility of deploying ensemble deep learning models within a real-world web application framework. While further enhancements such as dataset expansion, cloud deployment, and advanced model optimization can improve performance, the current implementation demonstrates the practical effectiveness of deep learning in medical image analysis and intelligent healthcare systems.

6.2 FUTURE ENHANCEMENT

Although the current system performs effectively, several improvements can be implemented in future versions to enhance performance and usability:

- 1. Performance Optimization**

Further fine-tuning of deep learning models and incorporation of advanced ensemble strategies can improve classification accuracy.

- 2. Dataset Expansion**

Increasing the dataset size and ensuring better class balance can enhance generalization capability.

- 3. Advanced Preprocessing Techniques**

Implementing image augmentation, contrast enhancement, and noise reduction techniques may improve prediction reliability.

- 4. Deployment on Cloud Platform**

Hosting the system on a cloud environment would allow real-time accessibility and scalability.

5. Mobile Application Integration

Developing a mobile-based version would increase accessibility for remote users and rural healthcare environments.

6. Clinical Validation

Collaborating with dermatologists for real-time validation can improve reliability and credibility.

7. Real-time Camera Integration

Integration with live camera input for instant disease prediction.

8. Improved Explainability Modules

Incorporating additional explainable AI techniques such as SHAP or LIME for deeper model transparency.

7. REFERENCES

7.1 BOOK REFERENCES

1. Aurélien Geron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd Edition, O'Reilly Media, 2019.
2. Charu C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Springer, 2018.
3. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Francois Chollet, *Deep Learning with Python*, Manning Publications, 2018.
5. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
6. Simon Haykin, *Neural Networks and Learning Machines*, 3rd Edition, Pearson Education, 2009.

7.2 WEB REFERENCES

1. <https://www.dermnetnz.org>
2. <https://pytorch.org/docs>
3. <https://flask.palletsprojects.com>
4. <https://www.sqlite.org/docs.html>
5. <https://keras.io/api/preprocessing/image/>

7.3 JOURNAL REFERENCES

1. M. Giavina-Bianchi, et al., “Explainability agreement between dermatologists and GradCAM visualizations,” *Frontiers in Medicine*, vol. 10, 2023, doi: <https://doi.org/10.3389/fmed.2023.1241484>
2. K. Jeyageetha, K. Vijayalakshmi, S. Suresh, and A. Bhuvanesh, “Multi-skin disease classification using hybrid deep learning model,” *Journal of Medical Systems*, vol. 49, 2025, doi: <https://doi.org/10.1177/09287329241312628>
3. J. M. Shivaram, et al., “Ensemble method of pre-trained models for skin lesion classification,” *Applied Sciences*, vol. 15, no. 24, p. 13083, 2025, doi: <https://doi.org/10.3390/app152413083>
4. S. M. Thwin and H.-S. Park, “Skin lesion classification using a deep ensemble model,” *Applied Sciences*, vol. 14, no. 13, p. 5599, 2024, doi: <https://doi.org/10.3390/app14135599>

5. M. Wei, S. Wu, and X. Shen, “Electronic fusion model of DenseNet and attention mechanisms for skin disease classification,” *Electronics*, vol. 12, no. 2, p. 438, 2023, doi: <https://doi.org/10.3390/electronics12020438>

8. APPENDIX

8.1 SOURCE CODE

`_init.py_`

```
from flask import Flask
from flask_login import LoginManager
from app.utils.db import init_db
import os

login_manager = LoginManager()

def create_app():
    app = Flask(__name__)
    app.secret_key = "dev-secret-key-change-this" # TODO: Use environment variable

    # Init DB
    init_db()

    # Init Login Manager
    login_manager.init_app(app)
    login_manager.login_view = "auth.login"

    # Register Blueprints
    from app.routes_auth import auth_bp
    from app.routes_main import main_bp
    from app.routes_admin import admin_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)
    app.register_blueprint(admin_bp)
```

```
from flask import session
from app.utils.translations import get_translation
```

```
@app.context_processor
def inject_get_text():
    def get_text(key):
        lang = session.get('lang', 'en')
        return get_translation(key, lang)
    return dict(get_text=get_text)

return app
```

disease_info.py

```
# Disease Information for 19-Class Merged Skin Disease Dataset
# This file contains labels, descriptions, and medical advice for all disease categories
LABELS = [
    "Acne and Rosacea",
    "Bacterial Infection",
    "Contact Dermatitis",
    "Eczema",
    "Fungal Infection",
    "Hair Loss",
    "Herpes & STDs",
    "Infestations & Bites",
    "Lupus and Connective Tissue Disease",
    "Malignant Lesions",
    "Melanoma & Nevi",
    "Nail Disease",
    "Pigmentation Disorders",
    "Psoriasis & Lichen Planus",
    "Seborrheic Keratoses and other Benign Tumors",
    "Systemic Disease",
```

"Urticaria",
"Vascular Disorders",
"Viral Infection"

]

DESCRIPTIONS = {

"Acne and Rosacea": "Acne is caused by blocked pores and inflammation, appearing as pimples, blackheads, and cysts. Rosacea is a chronic condition causing facial redness, visible blood vessels, and sometimes acne-like bumps.",

"Bacterial Infection": "Bacterial skin infections including cellulitis (red, swollen, painful skin), impetigo (red sores with honey-colored crusts), folliculitis, and other infections caused by bacteria like staph or strep.",

"Contact Dermatitis": "Allergic or irritant reactions from contact with substances like poison ivy/oak, chemicals, metals, or cosmetics. Causes itchy, red, sometimes blistering rashes at the contact site.",

"Eczema": "A group of conditions causing inflamed, itchy, cracked, and rough skin. Can be triggered by allergens, irritants, stress, or genetic factors. Often chronic with flare-ups.",

"Fungal Infection": "Fungal infections including ringworm (tinea), athlete's foot, jock itch, yeast infections (candidiasis), and other fungal colonization of skin, causing itchy, scaly, circular patches.",

"Hair Loss": "Various conditions causing hair loss including alopecia areata (autoimmune patchy hair loss), androgenic alopecia (pattern baldness), telogen effluvium, and other hair/scalp disorders.",

"Herpes & STDs": "Sexually transmitted infections affecting the skin including herpes simplex (cold sores, genital herpes), genital warts (HPV), syphilis rashes, and other STD manifestations.",

"Infestations & Bites": "Skin conditions from parasites and insect bites including scabies (mites causing intense itching), Lyme disease (tick-borne with bull's-eye rash), bed bugs, lice, and other infestations.",

"Lupus and Connective Tissue Disease": "Autoimmune diseases affecting skin and connective tissue. Lupus causes butterfly-shaped facial rashes, photosensitivity, and other skin changes. Includes dermatomyositis and scleroderma.",

"Malignant Lesions": "Precancerous and cancerous skin lesions including actinic keratosis (rough, scaly patches from sun damage), basal cell carcinoma, squamous cell carcinoma, and other malignant growths.",

"Melanoma & Nevi": "Includes melanoma (most serious skin cancer requiring urgent treatment), benign moles (nevi), and atypical moles requiring monitoring. Check using ABCDE rule (Asymmetry, Border, Color, Diameter, Evolving).",

"Nail Disease": "Nail disorders including fungal infections (onychomycosis causing discoloration and thickening), psoriatic nails, ingrown nails, paronychia, and other conditions affecting nail structure and appearance.",

"Pigmentation Disorders": "Conditions affecting skin color including vitiligo (loss of pigment), melasma (dark patches), hyperpigmentation, hypopigmentation, and sun-induced pigmentation changes.",

"Psoriasis & Lichen Planus": "Chronic inflammatory conditions. Psoriasis causes scaly, red, silver patches often on elbows, knees, scalp. Lichen planus causes purplish, itchy, flat-topped bumps on skin or mucous membranes.",

"Seborrheic Keratoses and other Benign Tumors": "Non-cancerous skin growths. Seborrheic keratoses are waxy, wart-like, stuck-on appearing growths common in older adults. Includes skin tags, dermatofibromas, and other benign tumors.",

"Systemic Disease": "Skin manifestations of internal diseases including diabetes (diabetic dermopathy, acanthosis nigricans), liver disease (jaundice, spider angiomas), kidney disease, thyroid disorders, and other systemic conditions.",

"Urticaria": "Hives - raised, itchy, red or skin-colored welts on the skin caused by allergic reactions, medications, foods, infections, stress, or other triggers. Can be acute or chronic.",

"Vascular Disorders": "Abnormalities of blood vessels including hemangiomas, port-wine stains, spider angiomas, vasculitis (inflamed blood vessels causing purpura or ulcers), and other vascular malformations or tumors.",

"Viral Infection": "Viral skin infections including common warts, plantar warts, flat warts, molluscum contagiosum (flesh-colored dome-shaped bumps), and other viral-induced skin lesions."

}

FIRST_AID = {

"Acne and Rosacea": "Keep skin clean with gentle cleanser, use non-comedogenic products. For rosacea, avoid triggers (alcohol, spicy food, hot beverages, sun exposure). See dermatologist for persistent or severe cases for prescription treatments.",

"Bacterial Infection": "URGENT: See doctor immediately, especially for cellulitis. Antibiotics required. Keep area clean, avoid touching/spreading. Cover with clean bandage. Watch for spreading redness, fever, or worsening symptoms.",

"Contact Dermatitis": "Immediately wash area with soap and water to remove irritant. Apply cool compresses, calamine lotion, or hydrocortisone cream. Take antihistamines for itching. See doctor if severe, spreading, or involves face/genitals.",

"Eczema": "Moisturize frequently with thick, fragrance-free creams. Use mild, unscented soaps. Avoid known triggers. Apply cool compresses for itching. Over-the-counter hydrocortisone for mild flares. See doctor if infected, severe, or not improving.",

"Fungal Infection": "Keep affected area clean and dry. Apply over-the-counter antifungal creams (clotrimazole, miconazole). Change socks/underwear daily. Avoid sharing towels. See doctor if not improving in 2 weeks or spreading.",

"Hair Loss": "See dermatologist for proper diagnosis and treatment. Treatment depends on cause - may include medications (minoxidil, finasteride), steroid injections, or immunotherapy. Some conditions resolve spontaneously.",

"Herpes & STDs": "See doctor immediately for testing and treatment. Antiviral medications available for herpes. Practice safe sex. Inform partners. Follow prescribed treatment plan. Some STDs require antibiotics or specific therapies.",

"Infestations & Bites": "URGENT for Lyme disease: See doctor immediately for antibiotics within 72 hours. For scabies: Prescription cream needed, wash all bedding/clothes in hot water. For bites: Clean area, apply ice, take antihistamines.",

"Lupus and Connective Tissue Disease": "MEDICAL ATTENTION NEEDED: Requires rheumatologist care for systemic management. Avoid sun exposure, wear SPF 50+ sunscreen daily. Follow prescribed immunosuppressive or anti-inflammatory medications. Monitor for systemic symptoms.",

"Malignant Lesions": "URGENT: See dermatologist immediately for biopsy and treatment. Do not delay. Wear broad-spectrum SPF 50+ sunscreen daily. Avoid sun exposure. Early treatment is highly effective. May require surgical removal, cryotherapy, or other treatments.",

"Melanoma & Nevi": "URGENT if mole is changing: See dermatologist immediately for full skin exam and possible biopsy. Monitor all moles monthly using ABCDE rule. Photograph suspicious lesions. Avoid sun exposure, wear sunscreen. Early detection is critical.",

"Nail Disease": "See doctor or dermatologist for accurate diagnosis. Fungal infections require oral or topical antifungal medications for several months. Keep nails clean, dry, and trimmed. Avoid nail polish during treatment.",

"Pigmentation Disorders": "Protect from sun with SPF 50+ broad-spectrum sunscreen daily. For melasma, avoid hormonal triggers. See dermatologist for treatment options including topical creams (hydroquinone, tretinoin), chemical peels, or laser therapy.",

"Psoriasis & Lichen Planus": "See dermatologist for treatment plan. Options include topical corticosteroids, vitamin D analogs, light therapy, or systemic medications for severe cases. Moisturize regularly. Manage stress. Avoid scratching.",

"Seborrheic Keratoses and other Benign Tumors": "Generally harmless, no treatment needed unless irritated or for cosmetic reasons. Can be removed by dermatologist via cryotherapy, electrocautery, or shave excision if desired. Monitor for changes.",

"Systemic Disease": "MEDICAL ATTENTION NEEDED: Skin changes may indicate serious underlying condition. See doctor for comprehensive medical evaluation including blood work and organ function tests. Treat underlying systemic disease.",

"Urticaria": "Take oral antihistamines (cetirizine, loratadine) to relieve itching. Apply cool compresses. Avoid known triggers. SEEK EMERGENCY CARE if experiencing throat swelling, difficulty breathing, or severe widespread hives (possible anaphylaxis).",

"Vascular Disorders": "See dermatologist or vascular specialist for evaluation. Most vascular lesions are benign. Treatment options available for cosmetic concerns (laser, sclerotherapy). For vasculitis: URGENT medical attention needed for underlying cause.",

"Viral Infection": "Most viral infections resolve on their own within weeks to months. Over-the-counter wart treatments available. Avoid picking or scratching. See doctor for persistent, spreading, or painful lesions. May require cryotherapy or other removal methods."
}

evaluate_pytorch.py

```
import os  
import numpy as np
```

```

import torch
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

from app.utils.pytorch_model import load_ensemble, CLASS_NAMES, TRANSFORM,
DEVICE
from PIL import Image

TEST_DIR = "dataset/test" # Root test folder (19 class subfolders)

print("Loading Ensemble Model...")
model = load_ensemble()
print("Model Loaded Successfully.\n")

y_true = []
y_pred = []

print("Evaluating test dataset...\n")

for class_idx, class_name in enumerate(CLASS_NAMES):
    class_path = os.path.join(TEST_DIR, class_name)

    if not os.path.exists(class_path):
        print(f"Warning: Folder not found -> {class_path}")
        continue

    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)

        try:
            img = Image.open(img_path).convert("RGB")
            img_tensor = TRANSFORM(img).unsqueeze(0).to(DEVICE)

```

```

with torch.no_grad():
    predictions, _ = model.predict(img_tensor)

    pred_class = np.argmax(predictions)

    y_true.append(class_idx)
    y_pred.append(pred_class)

except Exception as e:
    print(f'Error processing {img_path}: {e}')

print("\n===== 19-CLASS CLASSIFICATION REPORT
=====\\n")

print(classification_report(y_true, y_pred, target_names=CLASS_NAMES))

accuracy = accuracy_score(y_true, y_pred)
precision_macro = precision_score(y_true, y_pred, average="macro")
recall_macro = recall_score(y_true, y_pred, average="macro")
f1_macro = f1_score(y_true, y_pred, average="macro")

print("-----")
print("Accuracy      :", round(accuracy, 4))
print("Precision (macro) :", round(precision_macro, 4))
print("Recall (macro)   :", round(recall_macro, 4))
print("F1-score (macro)  :", round(f1_macro, 4))
print("-----")

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(14, 12))
sns.heatmap(cm, annot=False, cmap="Blues")

plt.title("Confusion Matrix - 19 Class Ensemble Model", fontsize=16)

```

```

plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.xticks(np.arange(len(CLASS_NAMES)) + 0.5, CLASS_NAMES, rotation=90)
plt.yticks(np.arange(len(CLASS_NAMES)) + 0.5, CLASS_NAMES, rotation=0)

plt.tight_layout()
plt.show()
print("\nEvaluation Completed Successfully.")

```

models.py

```

from flask_login import UserMixin
from app.utils.db import get_connection
from app import login_manager

class User(UserMixin):
    def __init__(self, id, username, role, email, is_verified=0):
        self.id = id
        self.username = username
        self.role = role
        self.email = email
        self.is_verified = is_verified

    @staticmethod
    def get(user_id):
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT * FROM users WHERE id = ?", (user_id,))
        user = cur.fetchone()
        conn.close()
        if not user:
            return None

```

```

# Schema observed:
# 0:id, 1:username, 2:mobile, 3:role, 4:password_hash, 5:is_verified, 6:email

# We need to handle cases where email might be None (old users) or exist.
# mobile is at 2.

mobile = user[2]
role = user[3]
# is_verified at 5
is_verified = user[5] if len(user) > 5 else 1

# email at 6
email = user[6] if len(user) > 6 else None

# If email is missing, maybe fallback to mobile or empty?
# But our app now expects email.
final_email = email if email else mobile # Fallback for display if needed

return User(id=user[0], username=user[1], role=role, email=final_email,
is_verified=is_verified)

@staticmethod
def get_by_username(username):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM users WHERE username = ?", (username,))
    user = cur.fetchone()
    conn.close()

    if not user:
        return None

    role = user[3]
    is_verified = user[5] if len(user) > 5 else 1

```

```

        email = user[6] if len(user) > 6 else None

        return User(id=user[0], username=user[1], role=role, email=email,
is_verified=is_verified)

    @staticmethod
    def get_user_with_password(username):
        """Helper to get user with password hash for auth verification"""
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT * FROM users WHERE username = ?", (username,))
        user = cur.fetchone()
        conn.close()
        return user

@login_manager.user_loader
def load_user(user_id):
    return User.get(user_id)

```

prepare_dataset.py

```

import os
import shutil
import pandas as pd

# Paths (EDIT ONLY IF NEEDED)
CSV_PATH = r"C:\Users\Joels\Downloads\archive (4)\HAM10000_metadata.csv"
IMAGES_DIR = r"D:/Downloads/archive (4)/processed_images_dataset/processed_images"
OUTPUT_DIR = r"D:/Smart_Dermatology_AI/dataset"

# Load CSV
df = pd.read_csv(CSV_PATH)

```

```

# Create class folders
for label in df['dx'].unique():
    os.makedirs(os.path.join(OUTPUT_DIR, label), exist_ok=True)

# Move images
for _, row in df.iterrows():
    img_name = row['image_id'] + ".jpg"
    label = row['dx']

    src = os.path.join(IMGES_DIR, img_name)
    dst = os.path.join(OUTPUT_DIR, label, img_name)

    if os.path.exists(src):
        shutil.copy(src, dst)
print("Dataset organized successfully!")

```

routes_admin.py

```

from flask import Blueprint, render_template, redirect, url_for, flash, request
from flask_login import login_required, current_user
from app.utils.db import get_connection
from datetime import datetime

admin_bp = Blueprint('admin', __name__)

@admin_bp.route('/admin')
@login_required
def dashboard():
    if current_user.role != 'admin':
        return redirect(url_for('main.home'))

    conn = get_connection()
    cur = conn.cursor()

```

```

# Get stats
cur.execute("SELECT COUNT(*) FROM users")
total_users = cur.fetchone()[0]

cur.execute("SELECT COUNT(*) FROM predictions")
total_predictions = cur.fetchone()[0]

# Disease Breakdown
cur.execute("SELECT disease, COUNT(*) as count FROM predictions GROUP BY
disease ORDER BY count DESC")
disease_stats = cur.fetchall()
total_diseases = len(disease_stats)

# Gender Breakdown
cur.execute("""
    SELECT u.gender, COUNT(*) as count
    FROM predictions p
    JOIN users u ON p.username = u.username
    GROUP BY u.gender
""")
gender_stats = cur.fetchall()

# Enhanced Recent Logins (Username, Patient Name, Latest Disease)
cur.execute("""
    SELECT l.username, l.login_time, u.patient_name,
    (SELECT disease FROM predictions p WHERE p.username = l.username ORDER BY
id DESC LIMIT 1) as latest_disease
    FROM login_history l
    JOIN users u ON l.username = u.username
    ORDER BY l.id DESC LIMIT 10
""")
recent_logins = cur.fetchall()

# Search Functionality

```

```

search_query = request.args.get('search', '')
if search_query:
    # If searching, find users matching name or username and get their history
    cur.execute("""
        SELECT p.*, u.patient_name, u.gender
        FROM predictions p
        JOIN users u ON p.username = u.username
        WHERE u.patient_name LIKE ? OR u.username LIKE ?
        ORDER BY p.timestamp DESC
        """, (f'%{search_query}%', f'%{search_query}%'))
    search_results = cur.fetchall()
else:
    search_results = None

# Get unverified users
cur.execute("SELECT * FROM users WHERE is_verified = 0")
unverified_users = cur.fetchall()

# Get All Users
cur.execute("SELECT * FROM users")
all_users = cur.fetchall()

# Recent Predictions (General)
cur.execute("SELECT * FROM predictions ORDER BY id DESC LIMIT 50")
recent_predictions = cur.fetchall()

conn.close()

return render_template('admin/dashboard.html',
    admin_mode=True,
    total_users=total_users,
    total_predictions=total_predictions,
    total_diseases=total_diseases,
    disease_stats=disease_stats,

```

```

        gender_stats=gender_stats,
        recent_logins=recent_logins,
        search_results=search_results,
        search_query=search_query,
        unverified_users=unverified_users,
        all_users=all_users,
        recent_predictions=recent_predictions)

@admin_bp.route('/admin/verify/<int:user_id>')
@login_required
def verify_user(user_id):
    if current_user.role != 'admin':
        return redirect(url_for('main.home'))

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("UPDATE users SET is_verified = 1 WHERE id = ?", (user_id,))
    conn.commit()
    conn.close()

    flash('User verified successfully.')
    return redirect(url_for('admin.dashboard'))

@admin_bp.route('/admin/analytics')
@login_required
def analytics():
    """Admin analytics dashboard with filtering"""
    if current_user.role != 'admin':
        return redirect(url_for('main.home'))

    # Get filter parameters from query string
    date_from = request.args.get('date_from', "")
    date_to = request.args.get('date_to', "")
    country_filter = request.args.get('country', "")

```

```

state_filter = request.args.get('state', "")
disease_filter = request.args.get('disease', "")
year_filter = request.args.get('year', "")

conn = get_connection()
cur = conn.cursor()

# Build dynamic query with filters
query = """
    SELECT p.*, u.patient_name, u.gender, u.location, u.state, u.country
    FROM predictions p
    JOIN users u ON p.username = u.username
    WHERE 1=1
    """
params = []

if date_from:
    query += " AND DATE(p.timestamp) >= ?"
    params.append(date_from)

if date_to:
    query += " AND DATE(p.timestamp) <= ?"
    params.append(date_to)

if year_filter:
    query += " AND strftime('%Y', p.timestamp) = ?"
    params.append(year_filter)

if country_filter:
    query += " AND u.country = ?"
    params.append(country_filter)

if state_filter:
    query += " AND u.state = ?"

```

```

        params.append(state_filter)

    if disease_filter:
        query += " AND p.disease = ?"
        params.append(disease_filter)

    query += " ORDER BY p.timestamp DESC"

    cur.execute(query, params)
    filtered_predictions = cur.fetchall()

    # Get unique values for filter dropdowns
    cur.execute("SELECT DISTINCT country FROM users WHERE country IS NOT NULL
ORDER BY country")
    countries = [row[0] for row in cur.fetchall()]

    cur.execute("SELECT DISTINCT state FROM users WHERE state IS NOT NULL
ORDER BY state")
    states = [row[0] for row in cur.fetchall()]

    cur.execute("SELECT DISTINCT disease FROM predictions ORDER BY disease")
    diseases = [row[0] for row in cur.fetchall()]

    cur.execute("SELECT DISTINCT strftime('%Y', timestamp) as year FROM predictions
ORDER BY year DESC")
    years = [row[0] for row in cur.fetchall()]

    # Statistics
    total_filtered = len(filtered_predictions)

    conn.close()

    return render_template('admin/analytics.html',
        predictions=filtered_predictions,

```

```

countries=countries,
states=states,
diseases=diseases,
years=years,
total_filtered=total_filtered,
filters={
    'date_from': date_from,
    'date_to': date_to,
    'country': country_filter,
    'state': state_filter,
    'disease': disease_filter,
    'year': year_filter
})

```

routes_auth.py

```

from flask import Blueprint, render_template, redirect, url_for, request, flash, session,
make_response
from flask_login import login_user, logout_user, login_required
from werkzeug.security import generate_password_hash, check_password_hash
from app.utils.db import get_connection
from app.models import User
from app.utils.captcha_gen import generate_captcha_image
import datetime

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/captcha')
def captcha():
    """Generates a CAPTCHA image and returns it."""
    image_io, text = generate_captcha_image()
    session['captcha'] = text
    response = make_response(image_io.getvalue())

```

```

response.headers['Content-Type'] = 'image/png'
return response

@auth_bp.route('/login-selection')
def login_selection():
    return render_template('login_selection.html')

@auth_bp.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        captcha_input = request.form.get('captcha')

        # Verify CAPTCHA first
        stored_captcha = session.get('captcha')
        if not stored_captcha or not captcha_input or stored_captcha.upper() !=
captcha_input.upper():
            flash('Invalid CAPTCHA code. Please try again.')
            return render_template('auth.html')

        # Clear CAPTCHA from session after attempt to prevent reuse
        session.pop('captcha', None)

        user_data = User.get_user_with_password(username)

        if user_data:
            # ACTUAL SCHEMA observed:
            # 0:id, 1:username, 2:mobile, 3:role, 4:password_hash, 5:is_verified, 6:email

            role = user_data[3]
            stored_hash = user_data[4]
            is_verified = user_data[5] if len(user_data) > 5 else 1
            email = user_data[6] if len(user_data) > 6 and user_data[6] else user_data[2]

```

```

if role == 'admin':
    flash('Admins must use the Admin Login page.')
    return redirect(url_for('auth.admin_login'))

if stored_hash and check_password_hash(stored_hash, password):
    if not is_verified:
        flash('Your account is pending verification. Please wait for admin approval.')
        return render_template('auth.html')

    # Authentication Successful -> Login Directly (No OTP)
    user = User(id=user_data[0], username=user_data[1], role=role, email=email,
is_verified=is_verified)
    login_user(user)

    # Log login history
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("INSERT INTO login_history (username, login_time) VALUES (?,
?)",
                (username, datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
    conn.commit()
    conn.close()

    return redirect(url_for('main.home'))
else:
    flash('Invalid password')
else:
    flash('User not found')

return render_template('auth.html')

# Removed /verify-otp and /validate-otp routes as they are no longer needed.

```

```

@auth_bp.route('/admin/login', methods=['GET', 'POST'])
def admin_login():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        user_data = User.get_user_with_password(username)

        if user_data:
            # Schema: 0:id, 1:username, 2:mobile, 3:role, 4:pass, 5:is_verified, 6:email
            role = user_data[3]
            stored_hash = user_data[4]
            is_verified = user_data[5] if len(user_data) > 5 else 1

            if role != 'admin':
                flash('Access Denied. Admins only.')
                return render_template('admin/login.html')

            if stored_hash and check_password_hash(stored_hash, password):
                # Admin login directly
                # User model handles indices now because we updated it?
                # Actually User __init__ takes args. We need to pass them correctly.
                # User(id, username, role, email, is_verified)

                email = user_data[6] if len(user_data) > 6 else user_data[2]
                user = User(id=user_data[0], username=user_data[1], role=role, email=email,
is_verified=is_verified)
                login_user(user)
                return redirect(url_for('admin.dashboard'))
            else:
                flash('Invalid password')
        else:
            flash('Admin user not found')

```

```

return render_template('admin/login.html')

@auth_bp.route('/register', methods=['POST'])
def register():
    username = request.form.get('username')
    email = request.form.get('email')
    password = request.form.get('password')

    conn = get_connection()
    cur = conn.cursor()

    hashed_password = generate_password_hash(password)

    try:
        # DB schema has MOBILE as NOT NULL at index 2.
        # We must supply it. We'll supply a dummy value because we are transitioning to Email.
        # Schema: username, mobile, password_hash, role, is_verified, email
        # No, INSERT requires column names usually to be safe.
        cur.execute("INSERT INTO users (username, mobile, email, password_hash, role,
is_verified) VALUES (?, ?, ?, ?, ?, ?)",
                    (username, '0000000000', email, hashed_password, 'user', 1))
        conn.commit()
        flash('Registration successful! Please login.')
    except Exception as e:
        print(f'Error: {e}')
        # Improve error message handling
        err_msg = str(e)
        if "UNIQUE constraint failed: users.username" in err_msg:
            flash('Username already exists.')
        elif "UNIQUE constraint failed" in err_msg:
            flash('User already exists.')
        else:
            flash(f'Registration failed: {e}')
    finally:

```

```

        conn.close()

    return redirect(url_for('auth.login'))

@auth_bp.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('auth.login'))

```

routes_main.py

```

from flask import Blueprint, render_template, request, redirect, url_for, flash,
send_from_directory, current_app
from flask_login import login_required, current_user
import os
import numpy as np
from datetime import datetime
# PyTorch imports
import torch
from PIL import Image
from app.utils.db import get_connection
# Gradient CAM import might need adjustment based on where it is.
# Assuming I moved it to app.utils.gradcam
# But wait, gradcam.py might rely on other things. I will check imports inside gradcam.py if
it fails.
from app.utils.gradcam import make_gradcam_heatmap, save_and_overlay_gradcam
from fpdf import FPDF
import uuid
from app.utils.report_gen import ReportGenerator

main_bp = Blueprint('main', __name__)

```

```

# Import PyTorch model loader
from app.utils.pytorch_model import predict_image, load_ensemble

# Load PyTorch ensemble model at startup
try:
    model = load_ensemble()
    print("[SUCCESS] PyTorch ensemble models loaded successfully (EfficientNet-B4 +
DenseNet121)")
except Exception as e:
    print(f"Warning: Could not load PyTorch models")
    print(e)
    model = None

# Check if user has completed profile before accessing protected pages
@main_bp.before_request
def check_profile_completion():
    """Redirect to profile completion if user hasn't filled their details"""
    if current_user.is_authenticated:
        # Skip check for these endpoints
        exempt_endpoints = ['main.complete_profile', 'auth.logout', 'static']
        if request.endpoint and any(request.endpoint.startswith(ep) for ep in exempt_endpoints):
            return None

        # Check if profile is complete
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT profile_completed FROM users WHERE username = ?",
(current_user.username,))
        result = cur.fetchone()
        conn.close()

        if result and result[0] == 0:
            return redirect(url_for('main.complete_profile'))
    return None

```

```

@main_bp.route('/complete-profile', methods=['GET', 'POST'])
@login_required
def complete_profile():
    """Handle user profile completion"""
    if request.method == 'POST':
        patient_name = request.form.get('patient_name', "").strip()
        gender = request.form.get('gender')
        location = request.form.get('location', "").strip()
        state = request.form.get('state', "").strip()
        country = request.form.get('country', "").strip()

        # Validate required fields
        if not all([patient_name, gender, location, state, country]):
            return render_template('user/complete_profile.html', error="All fields are required")

        # Update user profile
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("""
            UPDATE users
            SET patient_name = ?, gender = ?, location = ?, state = ?, country = ?,
profile_completed = 1
            WHERE username = ?
            """, (patient_name, gender, location, state, country, current_user.username))
        conn.commit()
        conn.close()

        flash("Profile completed successfully!", "success")
        return redirect(url_for('main.diagnose'))

    return render_template('user/complete_profile.html')

# Import disease information for 19-class merged dataset

```

```
from app.disease_info import LABELS, DESCRIPTIONS, FIRST_AID
```

```
def get_severity(conf):  
    if conf >= 85: return "High Risk"  
    elif conf >= 60: return "Moderate Risk"  
    return "Low Risk"
```

```
@main_bp.route('/set_language/<lang>')  
def set_language(lang):  
    from flask import session  
    session['lang'] = lang  
    return redirect(request.referrer or url_for('main.index'))
```

```
@main_bp.route('/')  
def index():  
    return render_template('index.html')
```

```
@main_bp.route('/home')  
@login_required  
def home():  
    return render_template('user/home.html')
```

```
@main_bp.route('/diagnose', methods=['GET', 'POST'])  
@login_required  
def diagnose():  
    if request.method == 'POST':  
        if 'file' not in request.files:  
            flash('No file part')  
            return redirect(request.url)  
        file = request.files['file']  
        if file.filename == "":  
            flash('No selected file')  
            return redirect(request.url)
```

```

if file and model:

    # Save file
    filename = str(uuid.uuid4()) + ".jpg"
    upload_path = os.path.join(current_app.root_path, 'static', 'uploads', filename)
    file.save(upload_path)

    # Predict using PyTorch ensemble
    result = predict_image(upload_path)
    disease_pre = result['disease']
    confidence = result['confidence']

    # Determine severity based on confidence
    if confidence < 30.0:
        disease = "Uncertain Diagnosis"
        severity = "Low Confidence"
        desc = "The uploaded image does not match any skin condition in our database
with sufficient confidence. This could be normal skin or a condition not in our dataset."
        first_aid = "Please consult a certified dermatologist for a proper diagnosis and
evaluation."
    else:
        # Valid match
        disease = disease_pre
        if confidence > 85:
            severity = "High Confidence"
        elif confidence > 60:
            severity = "Moderate Confidence"
        else:
            severity = "Low to Moderate Confidence"

        desc = DESCRIPTIONS.get(disease, "No description available.")
        first_aid = FIRST_AID.get(disease, "Consult a dermatologist for specific advice.")

    # GradCAM - need to preprocess image for PyTorch
    from app.utils.pytorch_model import TRANSFORM, DEVICE

```

```

img_pil = Image.open(upload_path).convert('RGB')
img_tensor = TRANSFORM(img_pil).unsqueeze(0).to(DEVICE)
heatmap = make_gradcam_heatmap(img_tensor, model)

# save_and_overlay_gradcam saves as {original}_gradcam.jpg
gradcam_full_path = save_and_overlay_gradcam(upload_path, heatmap)
gradcam_filename = os.path.basename(gradcam_full_path)

# Save DB
conn = get_connection()
cur = conn.cursor()
cur.execute(
    "INSERT INTO predictions (username, disease, confidence, timestamp,
original_img, gradcam_img) VALUES (?, ?, ?, ?, ?, ?)",
    (current_user.username, disease, confidence, datetime.now().strftime("%Y-%m-%d
%H:%M:%S"), filename, gradcam_filename)
)
conn.commit()
conn.close()

return render_template('user/result.html',
    disease=disease,
    confidence=confidence,
    severity=severity,
    desc=DESCRIPTIONS.get(disease, "No description available."),
    first_aid=FIRST_AID.get(disease, "Consult a dermatologist for specific
advice."),
    original_img=filename,
    gradcam_img=gradcam_filename)

return render_template('user/diagnose.html')

@main_bp.route('/history')
@login_required

```

```

def history():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM predictions WHERE username = ? ORDER BY id
DESC", (current_user.username,))
    rows = cur.fetchall()
    conn.close()
    return render_template('user/history.html', history=rows)

@main_bp.route('/report/<filename>')
@login_required
def download_report(filename):
    # Re-generate PDF on the fly or serve if saved.
    # For simplicity, let's just implement a route that triggers PDF gen or serve existing.
    # The original app generated it on the fly.
    # We can pass params to this route to regenerate? Or save it during diagnosis.
    """Generate and download PDF report for a diagnosis"""
    # Get diagnosis details from database
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("""
        SELECT p.disease, p.confidence, p.original_img, p.gradcam_img,
            u.patient_name, u.gender, u.location, u.state, u.country
        FROM predictions p
        JOIN users u ON p.username = u.username
        WHERE p.username = ? AND (p.original_img = ? OR p.gradcam_img = ?)
        ORDER BY p.id DESC LIMIT 1
    """, (current_user.username, filename, filename))
    result = cur.fetchone()
    conn.close()

    if not result:
        flash("Report not found", "error")
        return redirect(url_for('main.history'))

```

```

disease, confidence, original_img, gradcam_img, patient_name, gender, location, state,
country = result

# Determine severity
severity = get_severity(confidence)

# Paths to images
original_path = os.path.join(current_app.root_path, 'static', 'uploads', original_img) if
original_img else None
gradcam_path = os.path.join(current_app.root_path, 'static', 'uploads', gradcam_img) if
gradcam_img else None

# Generate PDF
try:
    report_gen = ReportGenerator(current_app.root_path)
    pdf_filename =
f'Skin_Report_{current_user.username}_{datetime.now().strftime('%Y%m%d_%H%M%S')}
}.pdf'

# Determine output directory
output_dir = os.path.join(current_app.root_path, 'static', 'reports')

full_pdf_path = report_gen.generate(
    username=current_user.username,
    disease=disease,
    confidence=confidence,
    severity=severity,
    original_img_path=original_path,
    gradcam_img_path=gradcam_path,
    output_filename=pdf_filename,
    patient_name=patient_name,
    gender=gender,
    location=location,

```

```

        state=state,
        country=country
    )

    return send_from_directory(
        output_dir,
        pdf_filename,
        as_attachment=True
    )

except Exception as e:
    print(f"Error generating PDF: {e}")
    flash(f"An error occurred while generating the report: {str(e)}", "error")
    return redirect(url_for('main.history'))

@main_bp.route('/generate_pdf', methods=['POST'])
@login_required
def generate_pdf_route():
    # Receive data from form
    disease = request.form.get('disease')
    confidence = float(request.form.get('confidence'))
    severity = request.form.get('severity')
    original_img = request.form.get('original_img')
    gradcam_img = request.form.get('gradcam_img')

    # Get user profile data
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT patient_name, gender, location, state, country FROM users
WHERE username = ?",
                (current_user.username,))
    profile = cur.fetchone()
    conn.close()

```

```
patient_name, gender, location, state, country = profile if profile else (None, None, None,
None, None)
```

```
# Paths
```

```
base_path = os.path.join(current_app.root_path, 'static', 'uploads')
```

```
orig_path = os.path.join(base_path, original_img)
```

```
grad_path = os.path.join(base_path, gradcam_img)
```

```
# Generate ID
```

```
report_filename = f'report_{uuid.uuid4()}.pdf'
```

```
# Generate Report
```

```
try:
```

```
    generator = ReportGenerator(current_app.root_path)
```

```
    generator.generate(
```

```
        username=current_user.username,
```

```
        disease=disease,
```

```
        confidence=confidence,
```

```
        severity=severity,
```

```
        original_img_path=orig_path,
```

```
        gradcam_img_path=grad_path,
```

```
        output_filename=report_filename,
```

```
        patient_name=patient_name,
```

```
        gender=gender,
```

```
        location=location,
```

```
        state=state,
```

```
        country=country
```

```
)
```

```
report_dir = os.path.join(current_app.root_path, 'static', 'reports')
```

```
if not os.path.exists(os.path.join(report_dir, report_filename)):
```

```
    flash('Error: Report generation failed (file not found).')
```

```
    return redirect(request.referrer or url_for('main.home'))
```

```
return send_from_directory(report_dir, report_filename, as_attachment=True)
```

```
except Exception as e:
```

```
    print(f"PDF Generation Error: {e}")
```

```
    flash('An error occurred while generating the report.')
```

```
    return redirect(request.referrer or url_for('main.home'))
```

8.2 OUTPUT SCREENSHOTS

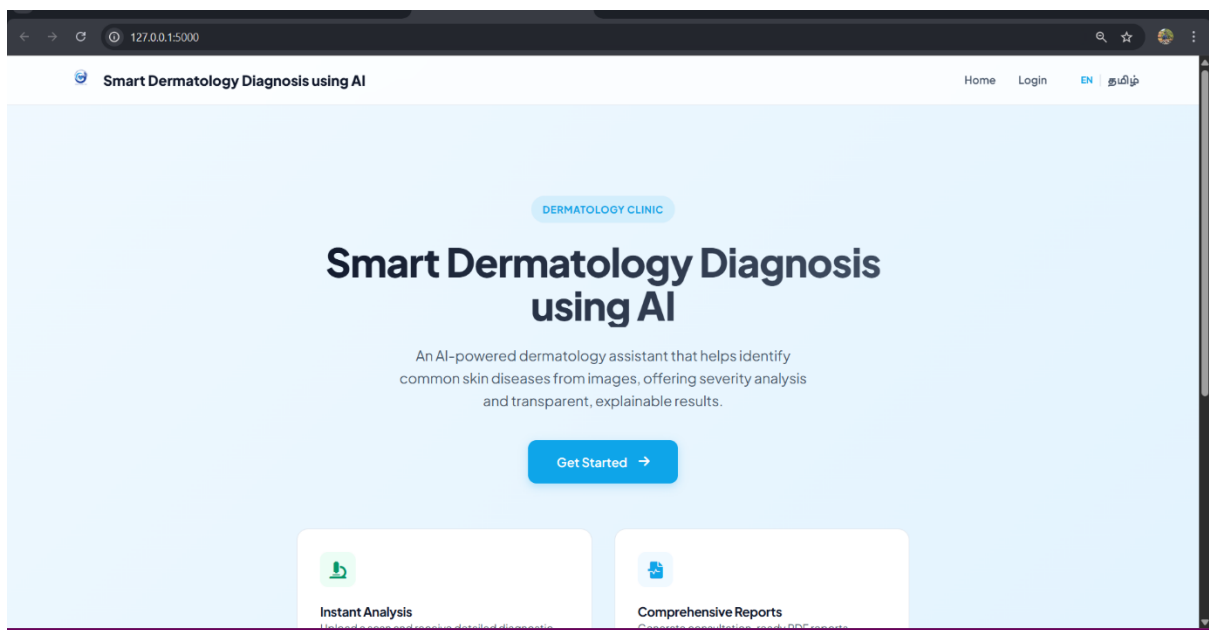


Figure 8.1 Smart Dermatology Diagnosis Web App Home Page

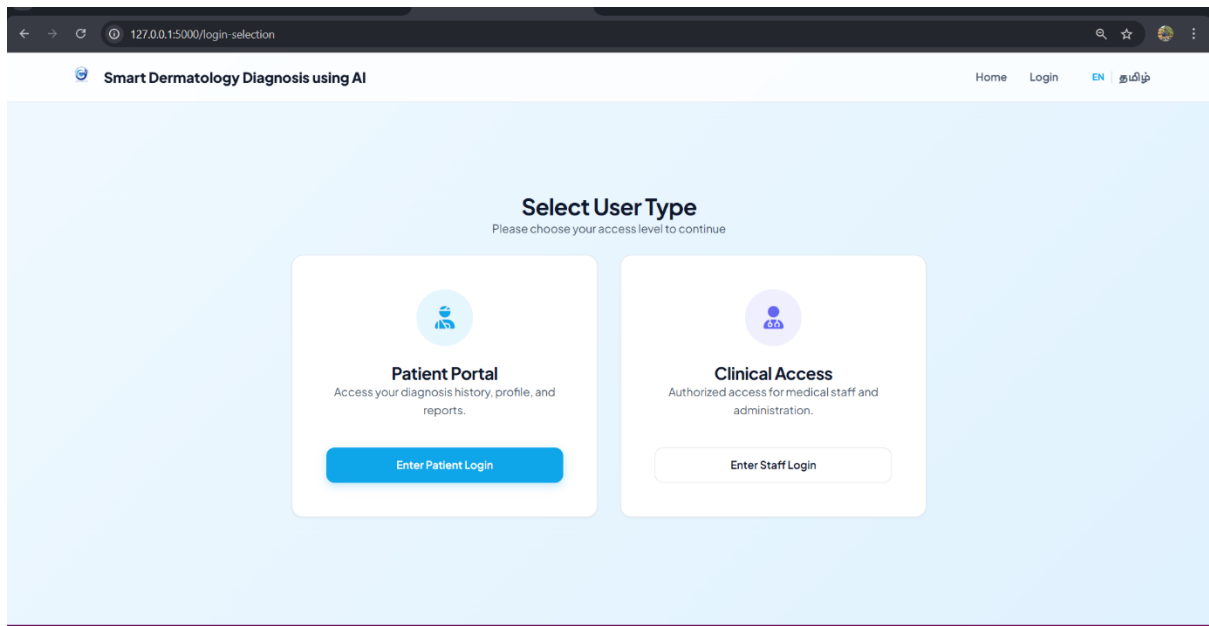


Figure 8.2 Smart Dermatology Diagnosis Web App Login Page

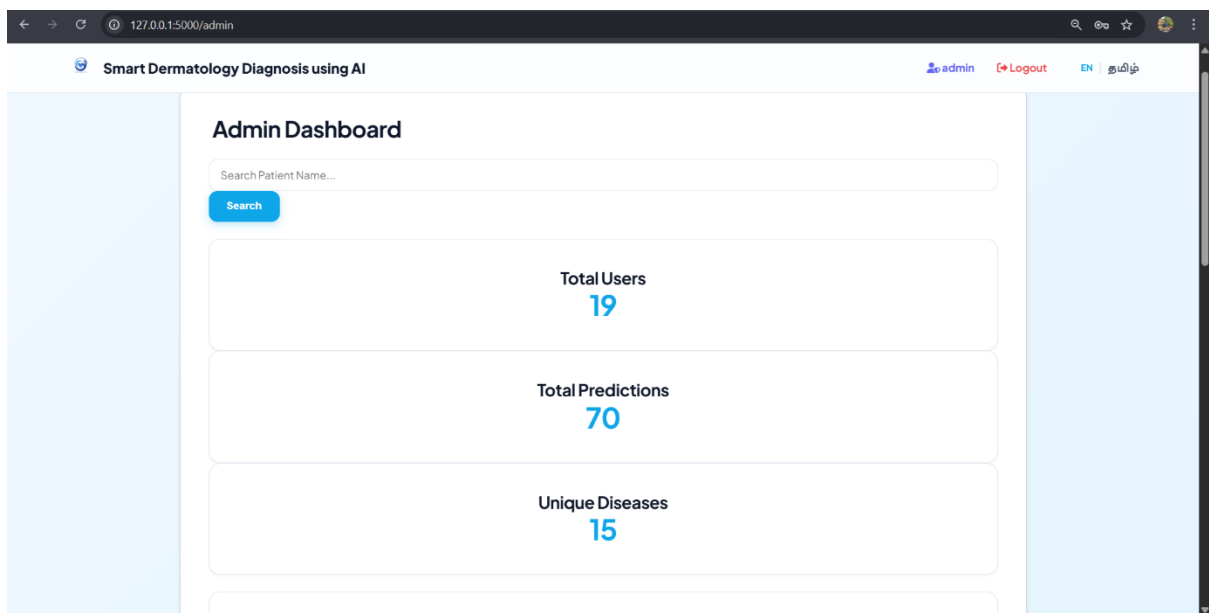


Figure 8.3 Smart Dermatology Diagnosis Web App Admin Dashboard

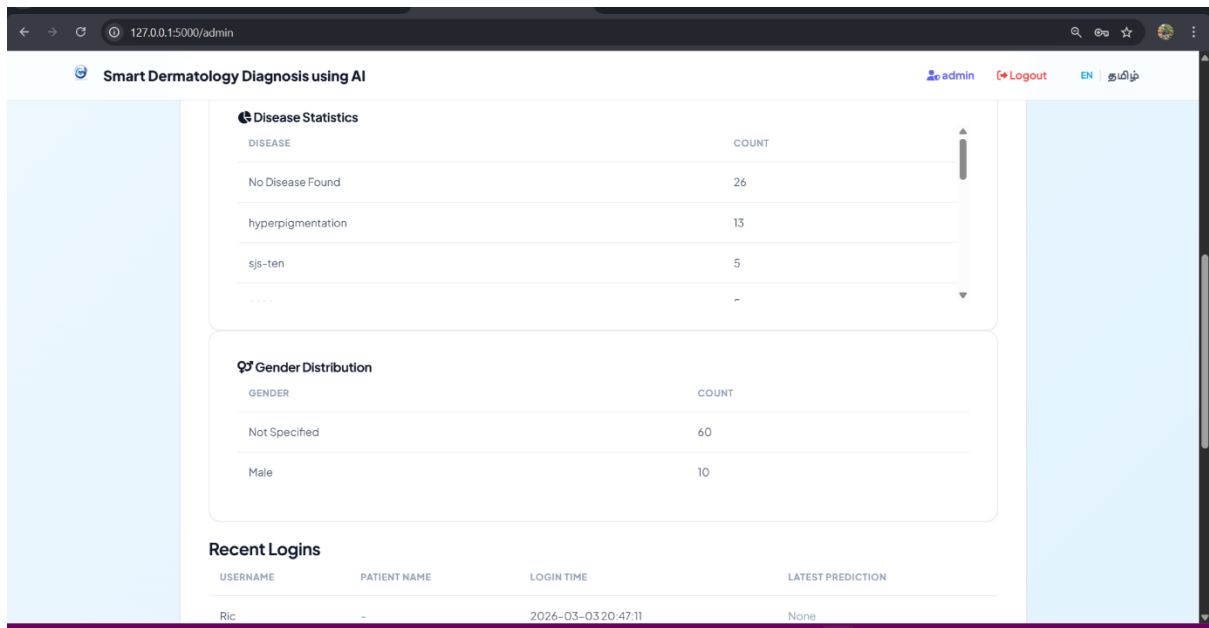


Figure 8.4 Smart Dermatology Diagnosis Web App Admin Dashboard (With Filters)

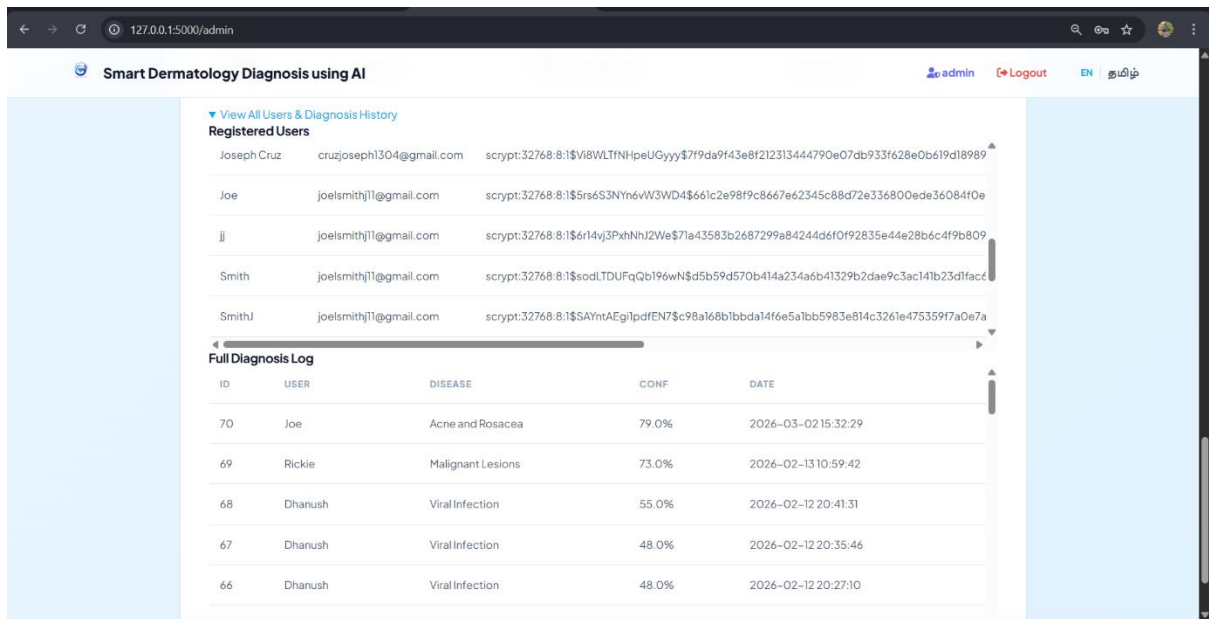


Figure 8.5 Smart Dermatology Diagnosis Web App Admin Dashboard (Viewing of users)

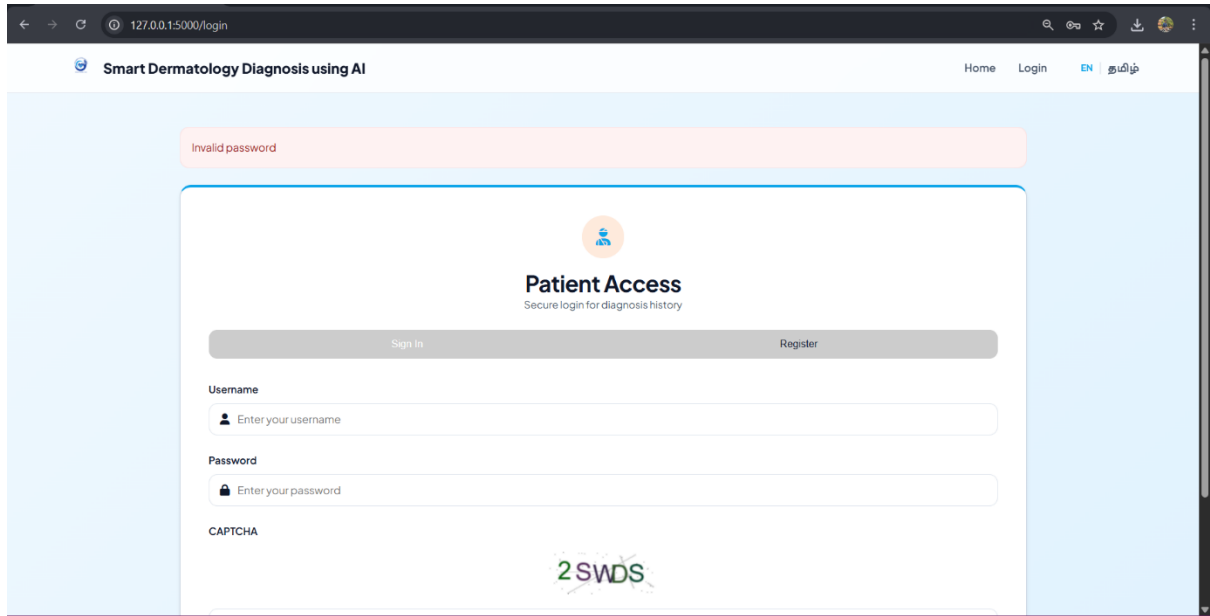


Figure 8.6 Smart Dermatology Diagnosis Web App Patient Login Page

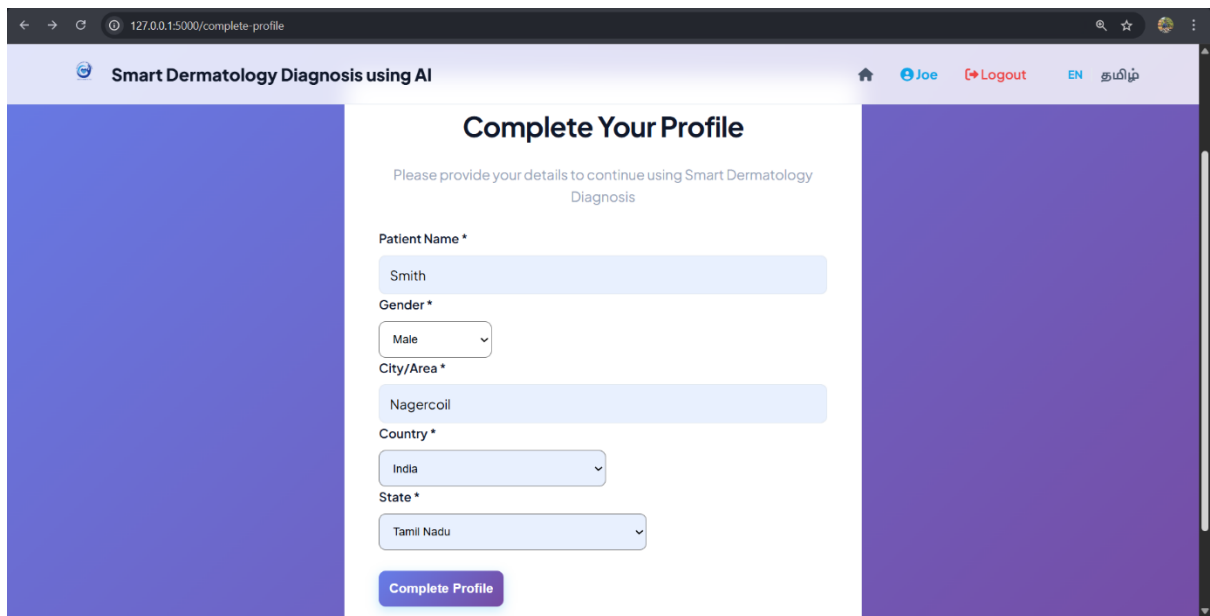


Figure 8.7 Smart Dermatology Diagnosis Web App Patient Profile Register Page

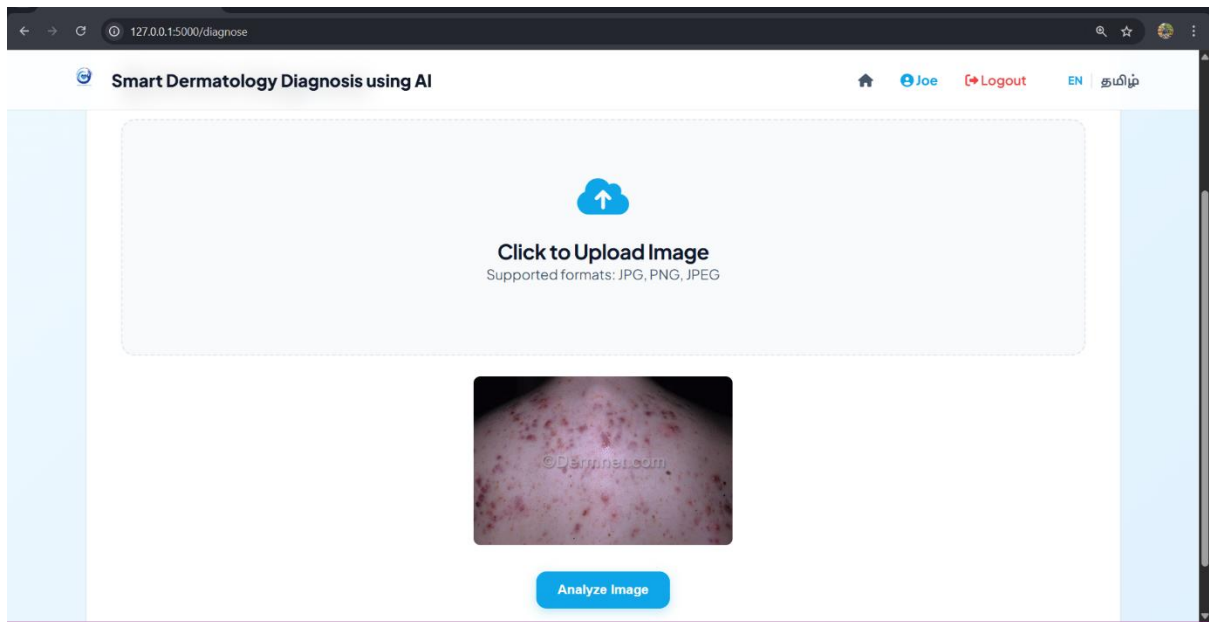


Figure 8.8 Smart Dermatology Diagnosis Web App - Uploading Skin Image

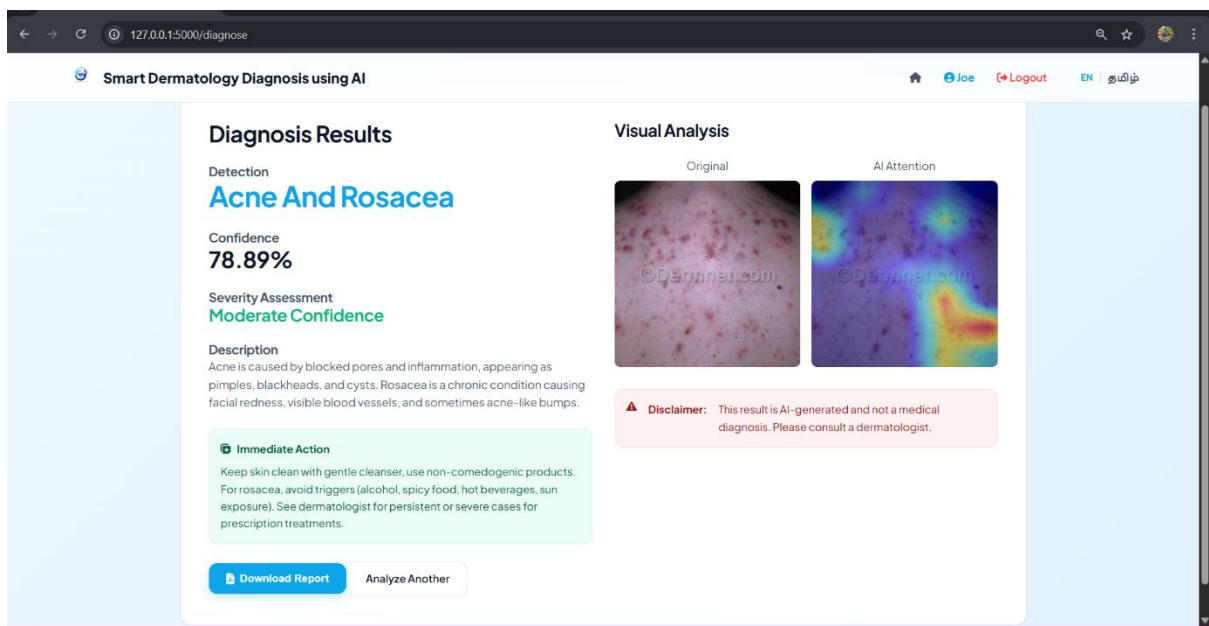


Figure 8.9 Smart Dermatology Diagnosis Web App – Diagnosis Page

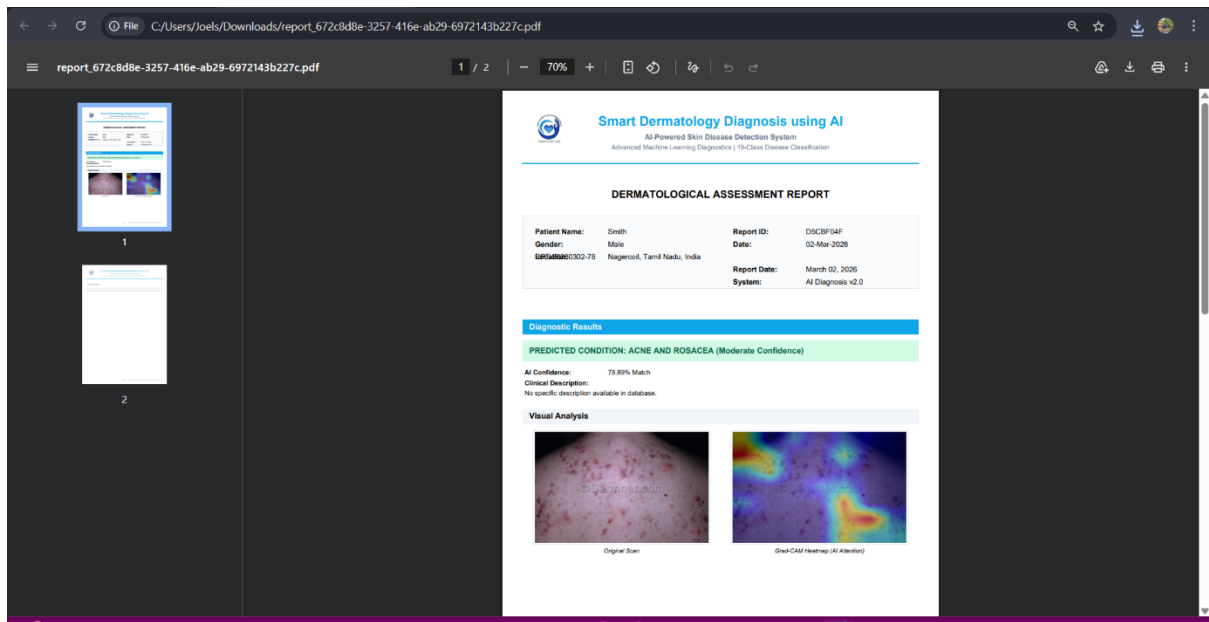


Figure 8.10 Smart Dermatology Diagnosis Web App – Diagnosis Report Generated as PDF